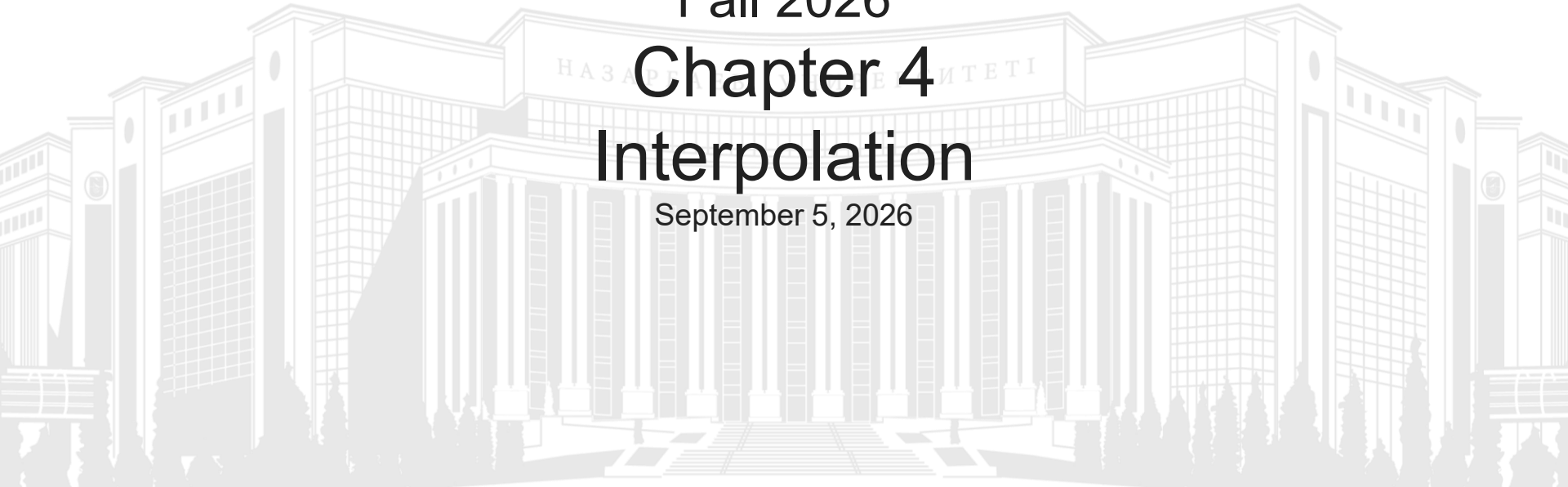


NAZARBAYEV UNIVERSITY

Numerical Methods with Applications
Math 351 – 1L, 2L

Fall 2026

Chapter 4
Interpolation

September 5, 2026

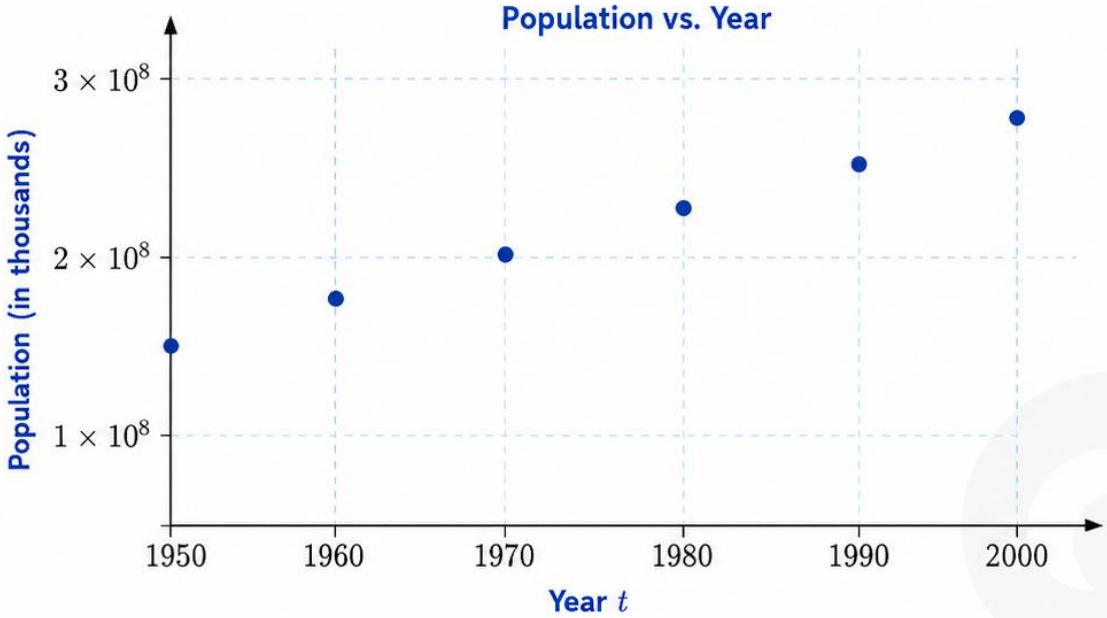
Population of the U.S.A. (1950–2000)

Year (t)	1950	1960	1970	1980	1990	2000
Population $P(t)$ (in thousands)	151,326	179,323	203,302	226,542	249,633	281,422

Discrete population data

The table lists the U.S. population every 10 years from 1950 to 2000. The scatter plot shows how the population $P(t)$ (in thousands) changes over time t (in years).

Observation:
The population increases nearly linearly over this period.



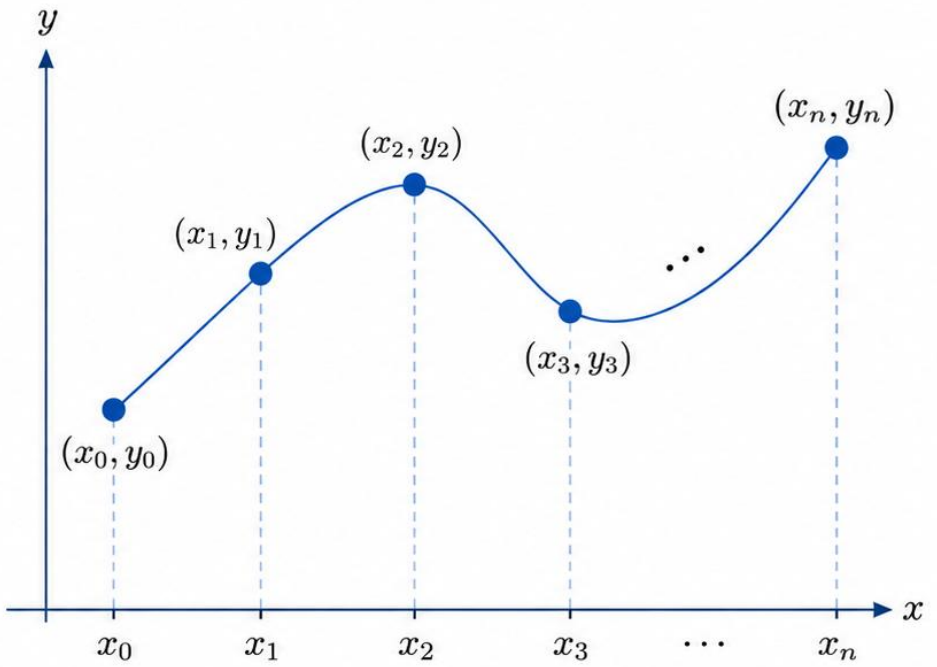
Use: These data can be used for interpolation, curve fitting, and numerical prediction of future population values.

Interpolating data

We are given a collection of data samples $\{(x_i, y_i)\}_{i=0}^{n}$.

- The $\{x_i\}_{i=0}^{n}$ are called the **abscissae** (singular: **abscissa**), the $\{y_i\}_{i=0}^{n}$ are called the **data values**.
- Want to find a function $v(x)$ which can be used to estimate sampled function for $x \neq x_i$:

Interpolation: $v(x_i) = y_i, \quad i = 0, 1, \ldots, n.$



Interpolating data

We are given a collection of **data samples** $\{(x_i, y_i)\}_{i=0}^{n}$.
We seek a **function** $v(x)$ such that $v(x_i) = y_i, \quad i = 0, 1, \ldots, n.$

Why interpolate?

We often have only discrete data, but we want information as if the underlying function were known continuously.

Discrete measurements or computed values are available,
but we want to understand the behavior between sample points.

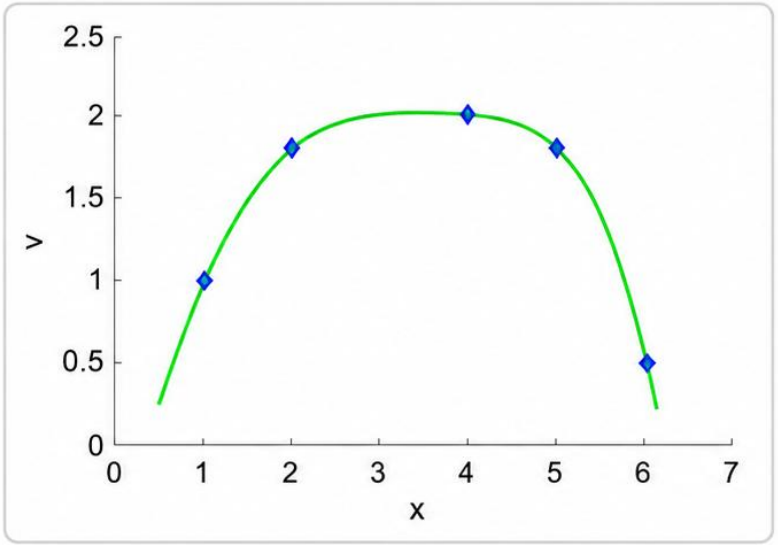
We may need to **plot, differentiate, or integrate** the data trend.

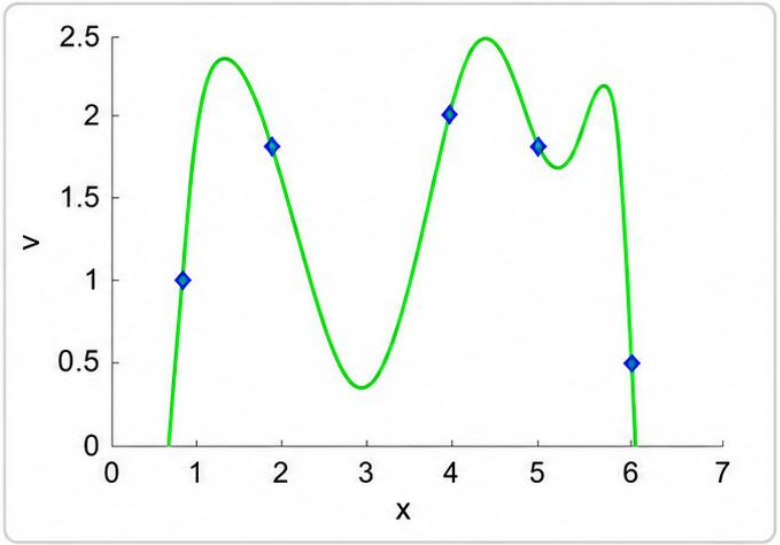
We may require a **simple and economical approximation** for the data.

Takeaway: Interpolation builds a **continuous model** from discrete data.

Interpolating data: wish list

- Want a **reasonable-looking** interpolant. Example:





The interpolant in the left figure looks more reasonable than the right one.

- If possible, v should be inexpensive to construct.
- If possible, $v(x)$ should be inexpensive to evaluate for a given x.

Interpolating functions

A function $f(x)$ may be given on an interval, $a \le x \le b$, explicitly or implicitly.

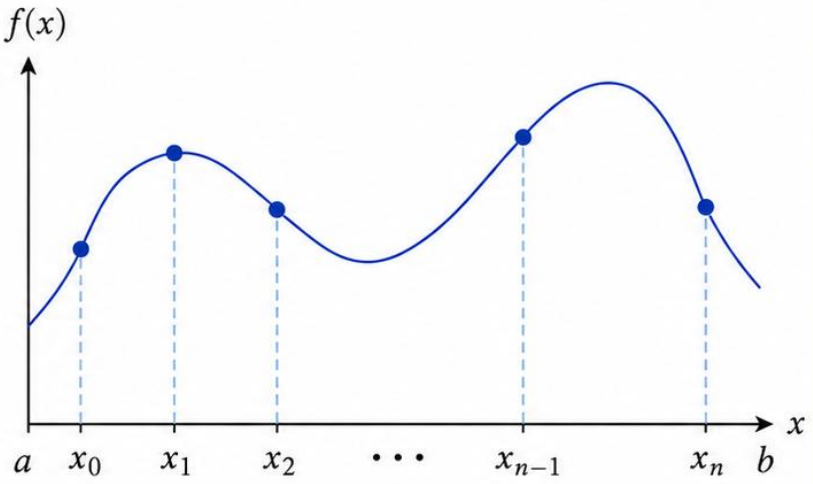
We want an interpolant $v(x)$ such that

$$v(x_i) = f(x_i), \quad i = 0, 1, \ldots, n,$$

at points $x_i \in [a, b]$.

The same algorithms as for interpolating data apply, but importantly here we may be able to:

- **choose the abscissae x_i;**
- **estimate interpolation error.**



sample points on a function

For functions, **node selection** and **error estimation** become important design choices.

Weierstrass Theorem Illustration

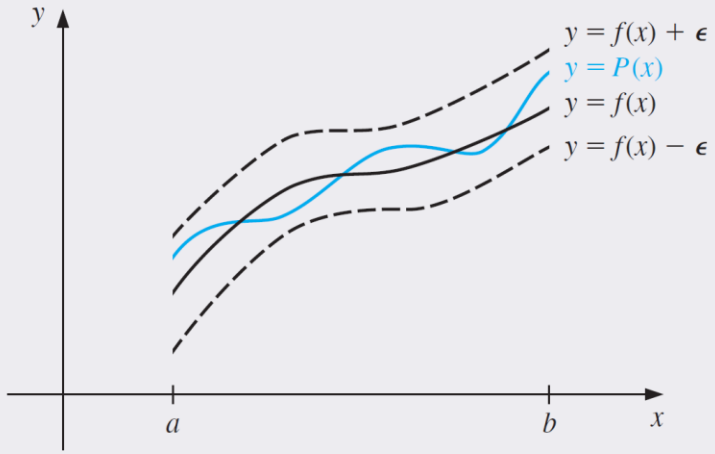
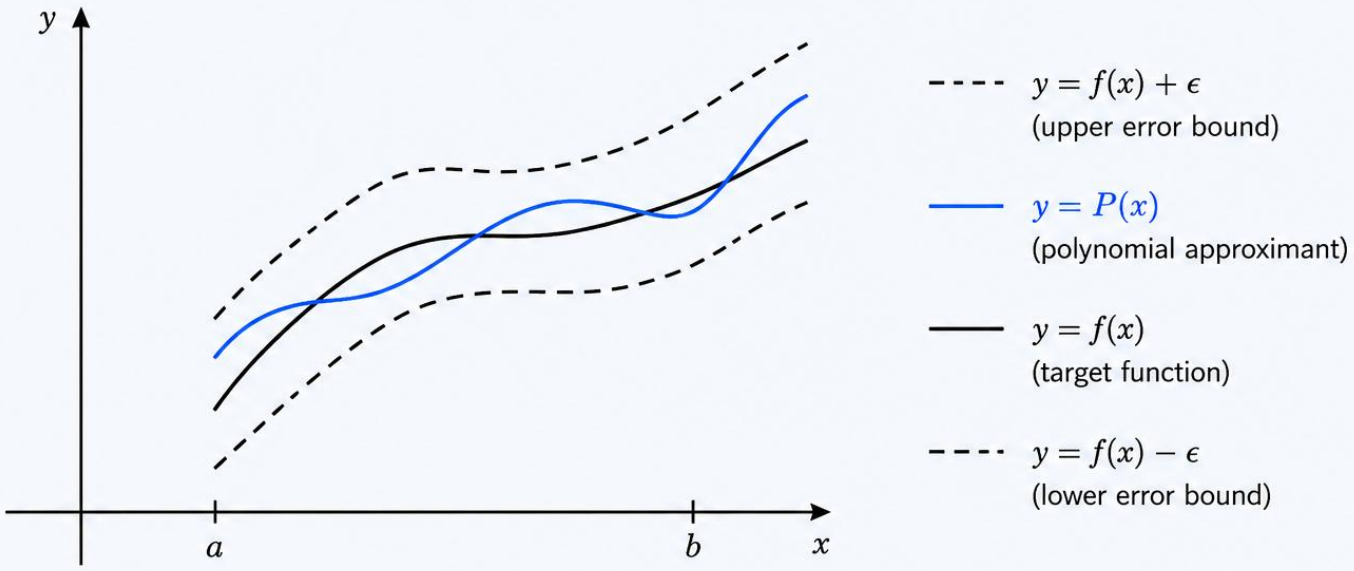


Figure: Figure 3.1

Weierstrass Theorem Illustration



The Weierstrass Approximation Theorem states that for any continuous function $f(x)$ on $[a, b]$ and any $\epsilon > 0$, there exists a polynomial $P(x)$ such that

$$|f(x) - P(x)| < \epsilon \quad \text{for all} \quad x \in [a, b].$$

Theorem 3.1 (Weierstrass Approximation Theorem)

Suppose that f is defined and continuous on $[a, b]$.

Then, for each $\epsilon > 0$, there exists a polynomial $P(x)$ with the property that

$$|f(x) - P(x)| < \epsilon,$$

for all $x \in [a, b]$.

In words: *Any continuous function on a closed interval can be approximated arbitrarily well by a polynomial.*

Taylor Polynomials: A Key Example

- A typical example for polynomial approximations is the Taylor polynomial.

$$P_n(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

↑ function value ↑ first-order term ↑ second-order term ↑ n-th order term

e^x

- For example, using Taylor polynomials at $x_0 = 0$ to approximate $f(x) = e^x$, we obtain

$$\begin{aligned}
P_0(x) &= 1, \\
P_1(x) &= 1 + x, \\
P_2(x) &= 1 + x + \frac{x^2}{2!}, \\
P_3(x) &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!}, \\
P_4(x) &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}, \\
P_5(x) &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!}, \\
&\vdots
\end{aligned}$$

As we include more terms, the Taylor polynomials provide better approximations to e^x near $x_0 = 0$.

10

Taylor Polynomials: Examples

1. Taylor polynomials for $f(x) = e^x$ about $x_0 = 0$

$$f(x) = e^x$$

$$\begin{aligned}
P_0(x) &= 1, \\
P_1(x) &= 1 + x, \\
P_2(x) &= 1 + x + \frac{x^2}{2}, \\
P_3(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6}, \\
P_4(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24}, \\
P_5(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120}, \quad \cdots
\end{aligned}$$

2. Taylor polynomials for $f(x) = \frac{1}{x}$ about $x_0 = 1$

approximate $f(3) = 1/3$. Since

$$f(x) = x^{-1}, \quad f'(x) = -x^{-2}, \quad f''(x) = (-1)^2 \cdot x^{-3}, \ \cdots$$

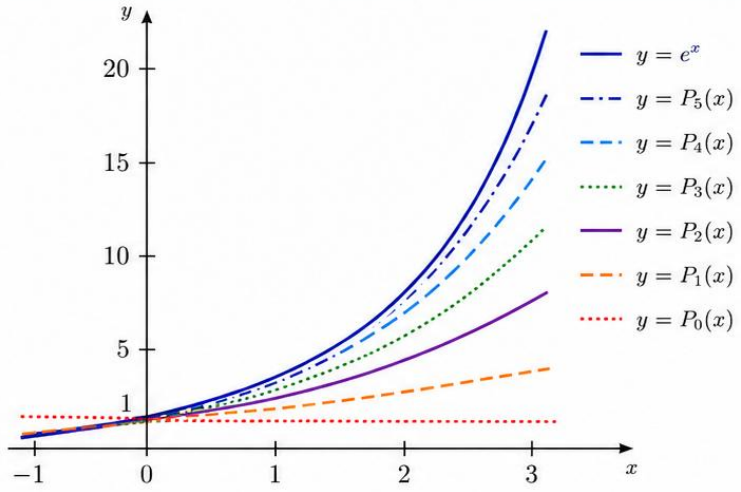
and, in general,

$$f^{(k)}(x) = (-1)^k k! x^{-k-1},$$

the Taylor polynomials are

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(1)}{k!} (x-1)^k = \sum_{k=0}^{n} (-1)^k (x-1)^k.$$

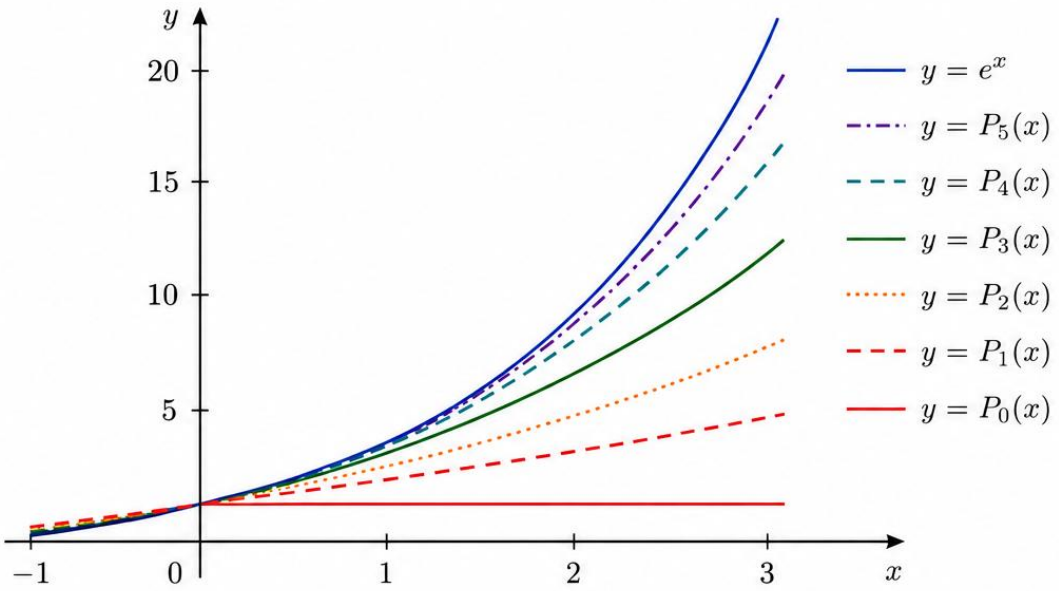
3. Graph: $f(x) = e^x$ and its Taylor polynomials about $x_0 = 0$



To approximate $f(3) = 1/3$ by $P_n(3)$ for increasing values of n, we obtain the values in the table below—rather a dramatic failure! When we approximate $f(3) = 1/3$ by $P_n(3)$ for larger values of n, the approximations become increasingly inaccurate.

n	0	1	2	3	4	5	6	7
$P_n(3)$	1	−1	3	−5	11	−21	43	−85

Taylor Polynomials About $x_0 = 0$



All polynomials are Taylor polynomials about $x_0 = 0$.

Key takeaway

Even for higher-degree polynomials, the error becomes progressively worse as we move away from the expansion point $x_0 = 0$.

At $x = 3$:

$f(3) = e^3 = 20.0855$	$P_1(3) = 4$	$P_2(3) = 8.5$	$P_3(3) = 13$	$P_4(3) = 16.375$

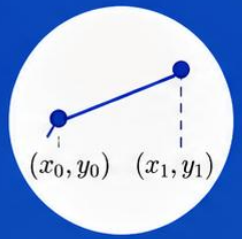
Remark on Taylor Polynomials

- For Taylor polynomials, **all information used in the approximation is concentrated at the single number x_0**, so these polynomials will generally give inaccurate approximations as we move away from x_0. This limits Taylor polynomial approximation to the situation in which approximations are needed only at numbers close to x_0.

- A good interpolation polynomial needs to **provide a relatively accurate approximation over an entire interval**, and Taylor polynomials do not generally do this.

- It is usually more efficient to develop methods that use **information spreaded at various points.**

Lagrange Interpolation Polynomials

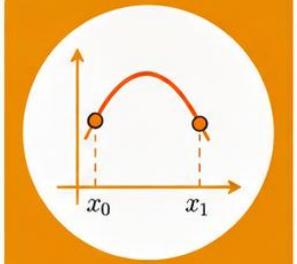


1 Suppose that a function $f(x)$ passes through two points (x_0, y_0) and (x_1, y_1). Define the following **linear Lagrange polynomials**

$$L_0(x) = \frac{x - x_1}{x_0 - x_1}, \qquad L_1(x) = \frac{x - x_0}{x_1 - x_0}.$$

2 It is easy to verify that

$$L_0(x_0) = 1, \qquad L_0(x_1) = 0,$$
$$L_1(x_0) = 0, \qquad L_1(x_1) = 1.$$



3 The **linear Lagrange interpolating polynomial** through (x_0, y_0) and (x_1, y_1) is

$$P(x) = y_0 L_0(x) + y_1 L_1(x) = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0}.$$

It can be verified that $P(x_0) = y_0$ and $P(x_1) = y_1$. **(Exercise)**

Example 2.

Determine the linear Lagrange interpolating polynomial that passes through the points $(2, 4)$ and $(5, 1)$.

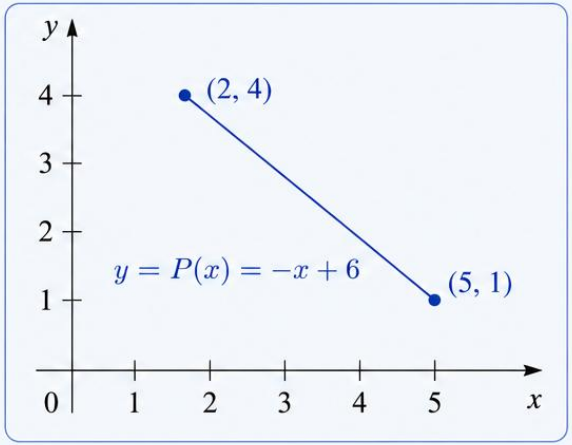
Solution

1. In this case, we have

$$L_0(x) = \frac{x - x_1}{x_0 - x_1} = \frac{x - 5}{2 - 5} = -\frac{1}{3}(x - 5), \quad \text{and} \quad L_1(x) = \frac{x - x_0}{x_1 - x_0} = \frac{x - 2}{5 - 2} = \frac{1}{3}(x - 2).$$

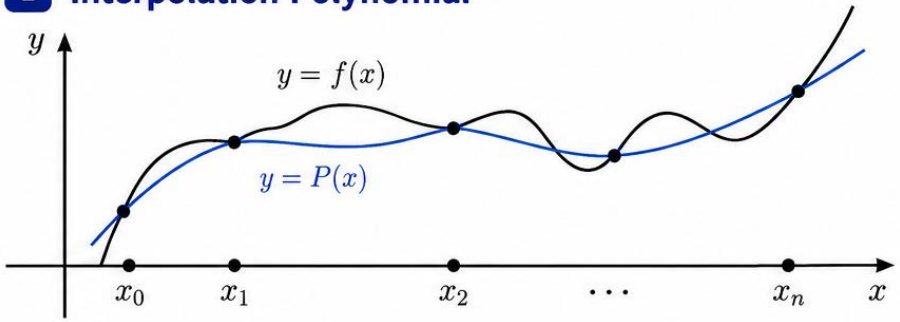
2. Then, the linear Lagrange interpolating polynomial is

$$\begin{aligned} P(x) &= y_0 L_0(x) + y_1 L_1(x) \\ &= 4\left(-\frac{1}{3}(x - 5)\right) + 1\left(\frac{1}{3}(x - 2)\right) \\ &= -\frac{4}{3}(x - 5) + \frac{1}{3}(x - 2) \\ &= -\frac{4}{3}x + \frac{20}{3} + \frac{1}{3}x - \frac{2}{3} \\ &= -x + 6. \end{aligned}$$



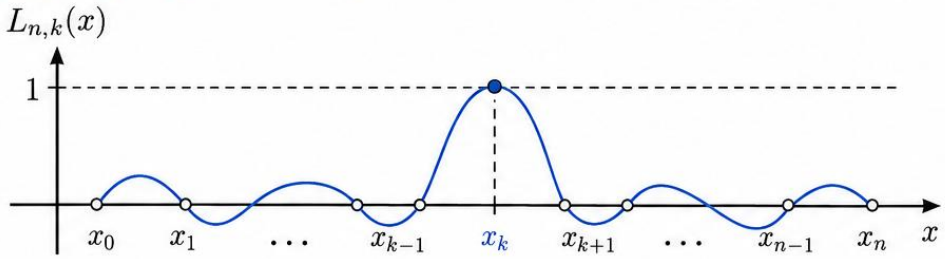
Lagrange Interpolation: Visualization

1 Interpolation Polynomial



- The interpolating polynomial $P(x)$ **passes through** the data points $(x_0, f(x_0)), (x_1, f(x_1)), \cdots, (x_n, f(x_n))$.
- It provides an **approximation** to $f(x)$ over the interval $[x_0, x_n]$.

2 Lagrange Basis Polynomials $L_{n,k}(x)$



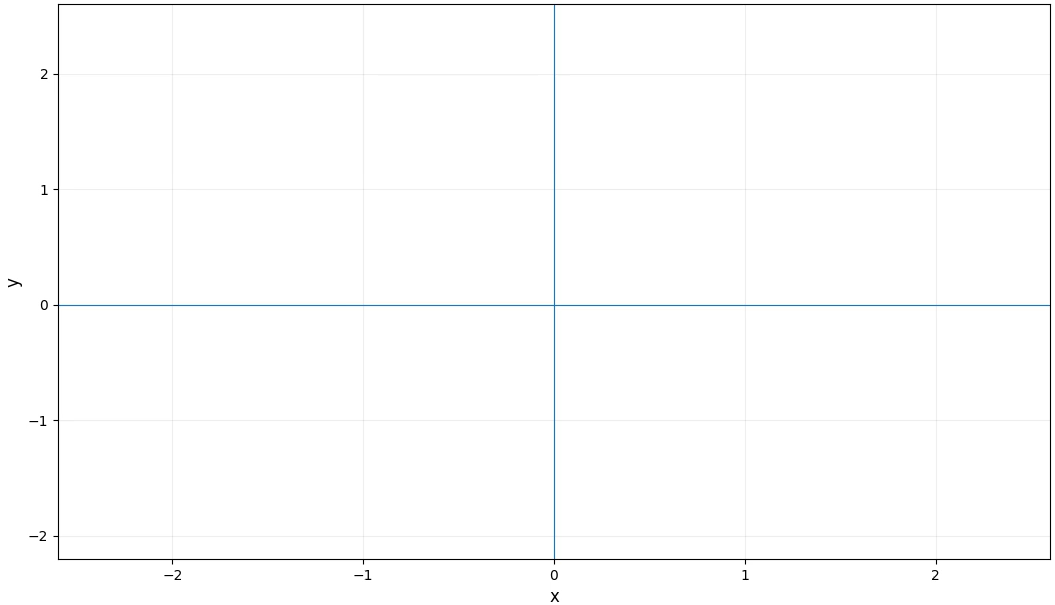
- $L_{n,k}(x_i) = 0 \quad \text{for} \quad i \neq k$
- $L_{n,k}(x_k) = 1$
- Each $L_{n,k}(x)$ **"peaks"** at x_k and crosses zero at all other nodes.

Key idea: The interpolating polynomial is a linear combination of the Lagrange basis polynomials:

$$P(x) = \sum_{k=0}^{n} f(x_k)\, L_{n,k}(x)$$

Lagrange Interpolation for n Data Points

Given $(x_0, y_0), \ldots, (x_{n-1}, y_{n-1})$, construct one polynomial through all points.



Theorem (3.2: n^{th} Lagrange Interpolating Polynomial)

If $x_0, x_1, \ldots, x_n$ are $n+1$ distinct numbers and f is a function whose values are given at these numbers, then **a unique polynomial** $P(x)$ of **degree at most** n exists with

$$f(x_k) = P(x_k), \quad \text{for each } k = 0, 1, \ldots, n.$$

1 Interpolation Polynomial

This polynomial is given by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) = \sum_{k=0}^{n} f(x_k)\, L_{n,k}(x).$$

2 Lagrange Basis Polynomials

where, for each $k = 0, 1, \ldots, n$,

$$L_{n,k}(x) = \frac{(x-x_0)(x-x_1)\cdots(x-x_{k-1})(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_0)(x_k-x_1)\cdots(x_k-x_{k-1})(x_k-x_{k+1})\cdots(x_k-x_n)} = \prod_{\substack{i=0 \\ i\neq k}}^{n} \frac{x-x_i}{x_k-x_i}.$$

Key features

- $P(x)$ has degree at most n.
- $L_{n,k}(x_k) = 1$ and $L_{n,k}(x_i) = 0$ for $i \neq k$.
- The polynomial $P(x)$ is unique.

Theorem 4.

If $x_0, x_1, \cdots, x_n$ are $n+1$ distinct numbers and f is a function whose values are given at these numbers, then a unique polynomial $P(x)$ of degree at most n exists with

$$f(x_k) = P(x_k), \qquad \text{for each } k = 0, 1, \cdots, n.$$

This polynomial is given by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) = \sum_{k=0}^{n} f(x_k)L_{n,k}(x),$$

where for each $k = 0, 1, \cdots, n$

$$L_{n,k}(x) = \frac{(x - x_0)\cdots(x - x_{k-1})(x - x_{k+1})\cdots(x - x_n)}{(x_k - x_0)\cdots(x_k - x_{k-1})(x_k - x_{k+1})\cdots(x_k - x_n)}$$

$$= \prod_{\substack{i=0 \\ i\neq k}}^{n} \frac{(x - x_i)}{(x_k - x_i)} \qquad \text{for each } k = 0, 1, \cdots, n.$$

Example 5.

Use the numbers (called **nodes**) $x_0 = 2$, $x_1 = 2.75$, and $x_2 = 4$ to find the second-degree Lagrange interpolating polynomial $P_2(x)$ for the function $f(x) = \frac{1}{x}$. Then use this polynomial to approximate $f(3) = \frac{1}{3}$.

Example 5.

Use the numbers (called **nodes**) $x_0 = 2$, $x_1 = 2.75$, and $x_2 = 4$ to find the second-degree Lagrange interpolating polynomial $P_2(x)$ for the function $f(x) = \frac{1}{x}$. Then use this polynomial to approximate $f(3) = \frac{1}{3}$.

Solution (1/2)

- The Lagrange basis polynomials for $x_0 = 2$, $x_1 = 2.75$, $x_2 = 4$ are

$$L_0(x) = \frac{(x-2.75)(x-4)}{(2-2.75)(2-4)} = \frac{2}{3}(x-2.75)(x-4).$$

$$L_1(x) = \frac{(x-2)(x-4)}{(2.75-2)(2.75-4)} = -\frac{16}{15}(x-2)(x-4).$$

$$L_2(x) = \frac{(x-2)(x-2.75)}{(2.75-2)(4-2.75)} = \frac{2}{5}(x-2)(x-2.75).$$

Solution (2/2)

- Also,

$$f(x_0) = f(2) = \frac{1}{2}, \quad f(x_1) = f(2.75) = \frac{4}{11}, \quad f(x_2) = f(4) = \frac{1}{4}.$$

- The second-degree Lagrange interpolating polynomial is

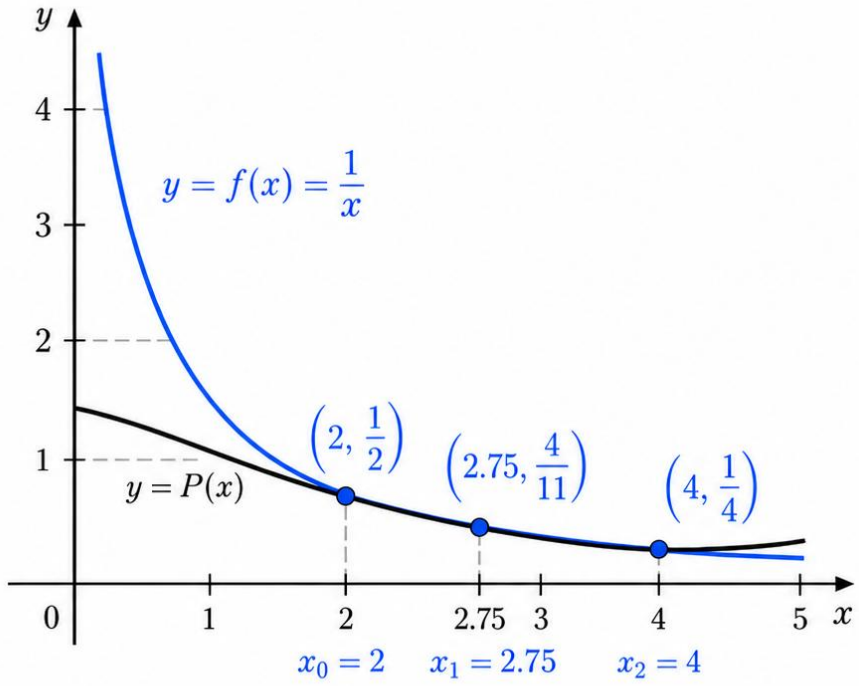
$$\begin{aligned} P_2(x) &= \sum_{k=0}^{2} f(x_k) L_k(x) \\ &= \frac{1}{2}\cdot\frac{2}{3}(x-2.75)(x-4) - \frac{4}{11}\cdot\frac{16}{15}(x-2)(x-4) + \frac{1}{4}\cdot\frac{2}{5}(x-2)(x-2.75) \\ &= \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}. \end{aligned}$$

- Use $P_2(x)$ to approximate $f(x)$ at $x = 3$

$$f(3) \approx P_2(3) = \frac{29}{88} = 0.32955 \qquad |f(3) - P_2(3)| = 0.03418$$

Example 5 – Interpolation of $f(x) = \frac{1}{x}$

- The black curve $P(x)$ is the **second-degree** Lagrange interpolating polynomial for $f(x) = \frac{1}{x}$ at the nodes $x_0 = 2,\ x_1 = 2.75,\ x_2 = 4$.



Key points

- **Blue curve:** $y = f(x) = \frac{1}{x}$ (exact function)
- **Black curve:** $y = P(x)$ (quadratic interpolant)
- They coincide at the nodes $x_0 = 2,\ x_1 = 2.75,\ x_2 = 4$.

Observation

The interpolating polynomial approximates $f(x) = \frac{1}{x}$ on $[2, 4]$, with the largest deviation near the endpoints.

Lagrange Interpolation Error Theorem

★ Theorem 3.3

Suppose $x_0, x_1, \ldots, x_n$ are distinct numbers in the interval $[a, b]$ and $f \in C^{n+1}[a, b]$. Then, for each $x \in [a, b]$, there exists at least one number $\xi(x)$ (generally unknown) between $\min\{x_0, x_1, \ldots, x_n\}$ and $\max\{x_0, x_1, \ldots, x_n\}$ and hence in (a, b), such that

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n),$$

where

- $P(x)$ is the Lagrange interpolating polynomial through the nodes $x_0, x_1, \ldots, x_n$ as given in Theorem 3.2.
- $\xi(x)$ depends on x in general and is not required to be unique.
- $f^{(n+1)}$ denotes the $(n+1)^{st}$ derivative of f.

Interpretation

The interpolation error equals the $(n+1)^{st}$ derivative of f at some interior point $\xi(x)$, divided by $(n+1)!$, times the product of distances from x to all nodes.

Comparison of Lagrange polynomials and Taylor polynomials

1 Taylor polynomial (about x_0)

Key idea

All known information is concentrated at the single point x_0.

- The nth-degree Taylor polynomial around x_0 has an error term of the form

$$R_n^{(\text{Taylor})}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1},$$

where ξ lies between x_0 and x.

2 Lagrange polynomial (interpolation at distinct nodes)

Key idea

Uses function values at distinct nodes $x_0, x_1, \ldots, x_n$ spread across the interval.

- The nth-degree Lagrange polynomial uses information at distinct numbers $x_0, x_1, \ldots, x_n$, and its error term has the form

$$R_n^{(\text{Lagrange})}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n),$$

where ξ lies in $(\min\{x_0, \ldots, x_n\}, \max\{x_0, \ldots, x_n\})$.

At a Glance

Information used

- **Taylor**: all at one point x_0
- **Lagrange**: at $n + 1$ distinct nodes

Error factor

- **Taylor**: $(x - x_0)^{n+1}$
- **Lagrange**: $(x - x_0)(x - x_1)\cdots(x - x_n)$ (product of $n + 1$ terms)

When to prefer

- **Taylor**: near x_0
- **Lagrange**: over a wider interval with data at several points

Example 7.

In Example 5, we found the 2nd-degree Lagrange polynomial $P_2(x)$ for $f(x) = \frac{1}{x}$ on $[2, 4]$ using the nodes $x_0 = 2$, $x_1 = 2.75$, $x_2 = 4$.

Determine the **error form** of this polynomial, and the **maximum error** when it is used to approximate $f(x)$ for $x \in [2, 4]$.

Solution (1/2)

1 Derivatives of $f(x) = \frac{1}{x}$

$$f'(x) = -x^{-2}, \qquad f''(x) = 2x^{-3}, \qquad f'''(x) = -6x^{-4}.$$

2 Error form (Theorem 6) on $[2, 4]$

$$f(x) - P_2(x) = \frac{f^{(3)}(\xi)}{3!}(x-2)(x-2.75)(x-4), \qquad \xi \in (2, 4).$$

$$\Rightarrow |f(x) - P_2(x)| = \left|\frac{f^{(3)}(\xi)}{3!} g(x)\right| = \frac{6\xi^{-4}}{6}|g(x)| = \underbrace{\frac{1}{\xi^4}}_{\leq \frac{1}{16} \text{ on } [2,4]}|g(x)| \leq \frac{1}{16}|g(x)|,$$

where

$$g(x) = (x-2)(x-2.75)(x-4) = x^3 - \frac{35}{4}x^2 + \frac{49}{2}x - 22.$$

Solution (2/2)

1 Find $\max_{x\in[2,4]} |g(x)|$

- Derivative:
$$g'(x) = 3x^2 - \frac{35}{2}x + \frac{49}{2} = \frac{1}{2}(3x-7)(2x-7).$$
- Critical points: $x_1 = \frac{7}{3}, \qquad x_2 = \frac{7}{2}.$
- Evaluate at critical points and endpoints:

x	2	$\frac{7}{3}$	$\frac{7}{2}$	4
$g(x)$	0	$\frac{25}{108}$	$-\frac{9}{16}$	0

2 Maximum error

$$\max_{x\in[2,4]} |f(x) - P_2(x)| \leq \frac{1}{16} \max_{x\in[2,4]} |g(x)| = \frac{1}{16} \cdot \frac{9}{16} = \mathbf{\frac{9}{256}} \approx 0.0351526.$$

Note In Example 5, the error at $x = 3$ is

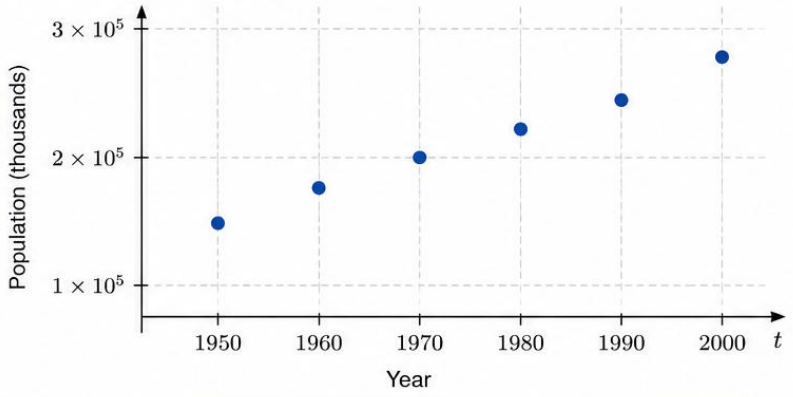
$$|f(3) - P_2(3)| = 0.03418 < 0.0351526 = \max_{2\leq x\leq 4} |f(x) - P_2(x)|.$$

Application Exercise

A census of the population of the United States is taken every 10 years.
The following table lists the population, in **thousands** of people, from 1950 to 2000,
and the data are also represented in the figure.

Year	1950	1960	1970	1980	1990	2000
Population (in thousands)	151,326	179,323	203,302	226,542	249,633	281,422

Data (Scatter Plot)



What we can do

In reviewing these data, we might ask whether they could be used to provide a reasonable estimate of the population, say, in **1975** or even in the year **2020**.

We can construct the **Lagrange interpolation** on these given data to generate a **fifth degree polynomial**.

Then use the interpolation polynomial for **prediction**.

Takeaway: Lagrange interpolation provides a polynomial model that passes exactly through all the data points and can be used to **estimate** the population at years **not included** in the data set.

Divided Differences

1. **Recall:** the n-th Lagrange interpolation $P_n(x)$ of the function $f(x)$ at $n+1$ distinct points $x_0, x_1, \ldots, x_n$.

Lagrange form: $$P_n(x) = \sum_{k=0}^{n} f(x_k) L_{n,k}(x)$$

Basis polynomials: $$L_{n,k}(x) = \prod_{\substack{i=0 \\ i \neq k}}^{n} \frac{x - x_i}{x_k - x_i}.$$

Key point

Each $L_{n,k}(x)$ depends on all nodes.

Adding a new node changes every basis polynomial.

Question: What if we have one more data point available $(x_{n+1}, f(x_{n+1}))$, then how to construct a new $n+1$-th degree interpolation $P_{n+1}(x)$?

Answer: We have to **abandon** all Lagrange polynomials $L_{n,k}(x)$, and reconstruct **new** Lagrange polynomials $L_{n+1,k}(x)$.

Question: Is there a more efficient way for adding more data points?

Motivation: Divided differences provide an efficient framework to update the interpolation when new data points are added.

Next: We introduce divided differences to address this challenge.

Divided Differences

Alternative representation of the interpolating polynomial

Although the interpolation polynomial $P_n(x)$ is unique, there are alternative representations that are useful in certain situations.

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}),$$

for appropriate constants $a_0, a_1, \cdots, a_n$.

Determine the first coefficients

$$\begin{aligned} P_n(x_0) = f(x_0) \quad &\Longrightarrow \quad a_0 = f(x_0) \\ P_n(x_1) = f(x_1) \quad &\Longrightarrow \quad a_1(x_1 - x_0) = f(x_1) - f(x_0) \\ &\Longrightarrow \quad a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0} \end{aligned}$$

Exercise

Verify that

$$a_2 = \frac{\dfrac{f(x_2) - f(x_1)}{x_2 - x_1} - \dfrac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$

We now introduce the divided-difference notation, **which will be very useful** in determining the values of a_i.

Definition 8 (Divided Difference)

1 The **zeroth divided difference** of the function f at x_i is

$$f[x_i] = f(x_i).$$

Zeroth divided difference
is just the function value.

2 The **first divided difference** of f at x_i and x_{i+1} is

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}.$$

Interpretation
Slope of the secant line through $(x_i, f(x_i))$ and $(x_{i+1}, f(x_{i+1}))$.

3 The **second divided difference** of f at x_i, x_{i+1} and x_{i+2} is

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}.$$

Interpretation
Change in slope.

4 In general, the k**-th divided difference** of f at $x_i, x_{i+1}, \ldots, x_{i+k}$ is

$$f[x_i, x_{i+1}, \ldots, x_{i+k}] = \frac{f[x_{i+1}, x_{i+2}, \ldots, x_{i+k}] - f[x_i, x_{i+1}, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Recursive
Built from lower-order divided differences.

Why useful? Divided differences allow us to compute the coefficients a_i of the interpolation polynomial efficiently and incrementally.

- **It can be seen that**

a_0 $$a_0 = f(x_0) = f[x_0]$$

a_1 $$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = f[x_0, x_1]$$

a_2 $$a_2 = \frac{\frac{f(x_2)-f(x_1)}{x_2-x_1} - \frac{f(x_1)-f(x_0)}{x_1-x_0}}{x_2 - x_0} = f[x_0, x_1, x_2]$$

- **In general, we have**

$$a_k = f[x_0, x_1, \ldots, x_k], \qquad \text{for all } k = 0, 1, \ldots, n.$$

Interpolation with Newton's Divided Difference

The Lagrange interpolation $P_n(x)$ of $f(x)$ at $x_0, x_1, \ldots, x_n$ can be written as

$$P_n(x) = f[x_0] + \sum_{k=1}^{n} f[x_0, x_1, \ldots, x_k](x - x_0)(x - x_1)\cdots(x - x_{k-1}).$$

- $f[x_0]$: Interpolates at x_0
- $f[x_0, x_1, \ldots, x_k]$: Divided difference (coefficient a_k)
- $(x - x_0)(x - x_1)\cdots(x - x_{k-1})$: Newton basis (uses previous nodes)

x	$f(x)$	First divided differences	Second divided differences	Third divided differences
x_0	$f[x_0]$			
		$f[x_0, x_1] = \dfrac{f[x_1] - f[x_0]}{x_1 - x_0}$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2] = \dfrac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$	
		$f[x_1, x_2] = \dfrac{f[x_2] - f[x_1]}{x_2 - x_1}$		$f[x_0, x_1, x_2, x_3] = \dfrac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3] = \dfrac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$	
		$f[x_2, x_3] = \dfrac{f[x_3] - f[x_2]}{x_3 - x_2}$		$f[x_1, x_2, x_3, x_4] = \dfrac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}$
x_3	$f[x_3]$		$f[x_2, x_3, x_4] = \dfrac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2}$	
		$f[x_3, x_4] = \dfrac{f[x_4] - f[x_3]}{x_4 - x_3}$		$f[x_2, x_3, x_4, x_5] = \dfrac{f[x_3, x_4, x_5] - f[x_2, x_3, x_4]}{x_5 - x_2}$
x_4	$f[x_4]$		$f[x_3, x_4, x_5] = \dfrac{f[x_4, x_5] - f[x_3, x_4]}{x_5 - x_3}$	
		$f[x_4, x_5] = \dfrac{f[x_5] - f[x_4]}{x_5 - x_4}$		
x_5	$f[x_5]$			

Divided Differences Table

x	$f(x)$	First divided differences	Second divided differences	Third divided differences
x_0	$f[x_0]$			
x_1	$f[x_1]$	$f[x_0, x_1] = \dfrac{f[x_1] - f[x_0]}{x_1 - x_0}$		
x_2	$f[x_2]$	$f[x_1, x_2] = \dfrac{f[x_2] - f[x_1]}{x_2 - x_1}$	$f[x_0, x_1, x_2] = \dfrac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$	
x_3	$f[x_3]$	$f[x_2, x_3] = \dfrac{f[x_3] - f[x_2]}{x_3 - x_2}$	$f[x_1, x_2, x_3] = \dfrac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$	$f[x_0, x_1, x_2, x_3] = \dfrac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$
x_4	$f[x_4]$	$f[x_3, x_4] = \dfrac{f[x_4] - f[x_3]}{x_4 - x_3}$	$f[x_2, x_3, x_4] = \dfrac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2}$	$f[x_1, x_2, x_3, x_4] = \dfrac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}$
x_5	$f[x_5]$	$f[x_4, x_5] = \dfrac{f[x_5] - f[x_4]}{x_5 - x_4}$	$f[x_3, x_4, x_5] = \dfrac{f[x_4, x_5] - f[x_3, x_4]}{x_5 - x_3}$	$f[x_2, x_3, x_4, x_5] = \dfrac{f[x_3, x_4, x_5] - f[x_2, x_3, x_4]}{x_5 - x_2}$

Remarks: The table is triangular. | The k-th divided differences use $k+1$ consecutive nodes. | This table is the basis for Newton's divided difference interpolation.

Example 9.

Compute a divided difference table for these function values:

x	3	1	5	6
$f(x)$	1	-3	2	4

Then determine the Newton interpolation polynomial.

Solution (1/2)

Arrange the table vertically to have

i	x_i	$f[x_i]$	$f[x_{i-1}, x_i]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, x_{i-2}, x_{i-1}, x_i]$
0	3	1			
1	1	-3	2		
2	5	2	$5/4$	$-3/8$	
3	6	4	2	$3/20$	$7/40$

Solution (2/2)

Note: The values marked in **red** and **blue** will be used for constructing the interpolating polynomial (**Newton coefficients**).

i	x_i	$f[x_i]$	$f[x_{i-1}, x_i]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, x_{i-2}, x_{i-1}, x_i]$
0	3	1			
1	1	-3	2		
2	5	2	$5/4$	$-3/8$	
3	6	4	2	$3/20$	$7/40$

Newton interpolation polynomial (using the highlighted coefficients):

$$\begin{aligned} P(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ &\quad + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2) \\ &= 1 + 2(x - 3) + -\frac{3}{8}(x - 3)(x - 1) + \frac{7}{40}(x - 3)(x - 1)(x - 5) \\ \Rightarrow &= \frac{1}{40}(7x^3 - 78x^2 + 301x - 350) \end{aligned}$$

Check: $P(3) = 1, \quad P(1) = -3, \quad P(5) = 2, \quad P(6) = 4.$ The polynomial matches all given data points.

Comparison of Lagrange and Newton

LAGRANGE INTERPOLATION

Requires less work (in many cases).

It is often recommended when the number of terms needed for sufficient accuracy is known in advance from theory or previous experience.

NEWTON INTERPOLATION

Easier to extend with new data.

Additional data points can be added to improve accuracy without re-doing the whole problem.

Key takeaway: Use Lagrange when the final number of terms is known.
Use Newton when data may be added incrementally.

Algorithm 3.2: NEWTON'S DIVIDED DIFFERENCE

To obtain the divided-difference coefficients of the interpolatory polynomial P on the $(n+1)$ distinct numbers $x_0, x_1, \ldots, x_n$ for the function f:

INPUT $x_0, x_1, \ldots, x_n$; values $f(x_0), f(x_1), \ldots, f(x_n)$ as $F_{0,0}, F_{1,0}, \ldots, F_{n,0}$.
OUTPUT the numbers $F_{0,0}, F_{1,1}, \ldots, F_{n,n}$ where

$$P_n(x) = F_{0,0} + \sum_{i=1}^{n} F_{i,i} \prod_{j=0}^{i-1} (x - x_j). \quad (F_{i,i} \text{ is } f[x_0, x_1, \ldots, x_i].)$$

Step 1 For $i = 1, 2, \ldots, n$

For $j = 1, 2, \ldots, i$

set $F_{i,j} = \dfrac{F_{i,j-1} - F_{i-1,j-1}}{x_i - x_{i-j}}$. $\quad (F_{i,j} = f[x_{i-j}, \ldots, x_i].)$

Step 2 OUTPUT $(F_{0,0}, F_{1,1}, \ldots, F_{n,n})$;

STOP.

Algorithm for Generating Divided Differences Table

- **Inputs**: **dataX** $= [x_0, x_1, x_2, \cdots, x_n]$, **dataY** $= [y_0, y_1, y_2, \cdots, y_n]$
- **Output**: $\mathbf{C} = [c_{ij}]$ is an $(n+1) \times (n+1)$ divided difference matrix.

$\mathbf{C}$ = divdifTable(**dataX**,**dataY**)

For $j = 1 : n$

  For $i = 0 : n - j$

$$c_{ij} = \frac{c_{i+1,j-1} - c_{i,j-1}}{x_{i+j} - x_i} \quad \text{(note: } c_{ij} = f[x_i, x_{i+1}, \cdots, x_{i+j}]\text{)}$$

  End

End

$$\begin{array}{cc|ccccccc} x_0 & y_0 & c_{00} & c_{01} & c_{02} & c_{03} & \cdots & c_{0,n-1} & c_{0n} \\ x_1 & y_1 & c_{10} & c_{11} & c_{12} & c_{13} & \cdots & c_{1,n-1} & \\ x_2 & y_2 & c_{20} & c_{21} & c_{22} & c_{23} & \cdots & & \\ \vdots & \vdots & \vdots & \vdots & \vdots & & & & \\ x_{n-1} & y_{n-1} & c_{n-1,0} & c_{n-1,1} & & & & & \\ x_n & y_n & c_{n0} & & & & & & \end{array}$$

note: $c_{i,0} = y_i$, for $i = 0, 1, \cdots, n$.

Table 3.10 Given data

x	$f(x)$
1.0	0.7651977
1.3	0.6200860
1.6	0.4554022
1.9	0.2818186
2.2	0.1103623

Table 3.10 Given data

x	$f(x)$
1.0	0.7651977
1.3	0.6200860
1.6	0.4554022
1.9	0.2818186
2.2	0.1103623

How the first few differences are computed

- First divided difference involving x_0 and x_1:

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{0.6200860 - 0.7651977}{1.3 - 1.0} = -0.4837057.$$

- Second divided difference involving x_0, x_1, x_2:

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{-0.5489460 - (-0.4837057)}{1.6 - 1.0} = -0.1087339.$$

- Second divided difference involving x_1, x_2, x_3:

$$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1} = \frac{-0.0494433 - (-0.1087339)}{1.9 - 1.3} = 0.0658784.$$

- Third divided difference involving x_0, x_1, x_2, x_3:

$$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{0.0658784 - (-0.1087339)}{1.9 - 1.0} = 0.0680685.$$

- Fourth divided difference involving x_0, x_1, x_2, x_3, x_4:

$$f[x_0, x_1, x_2, x_3, x_4] = \frac{f[x_1, x_2, x_3, x_4] - f[x_0, x_1, x_2, x_3]}{x_4 - x_0} = \frac{0.0018251 - 0.0680685}{2.2 - 1.0} = 0.0018251.$$

Solution The first divided difference involving x_0 and x_1 is

$$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{0.6200860 - 0.7651977}{1.3 - 1.0} = -0.4837057.$$

» The remaining first divided differences are found in a similar manner and are shown in the fourth column in Table 3.11.

Table 3.11 Divided-difference table

i	x_i	$f[x_i]$	$f[x_{i-1}, x_i]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-4}, \ldots, x_i]$
0	1.0	0.7651977				
1	1.3	0.6200860	−0.4837057			
2	1.6	0.4554022	−0.5489460	−0.1087339		
3	1.9	0.2818186	−0.5786120	−0.0494433	0.0658784	
4	2.2	0.1103623	−0.5715210	0.0118183	0.0680685	0.0018251

Key takeaway

- Each entry is computed from the two entries above it.
- The diagonal (colored) entries are the Newton coefficients used in the interpolation polynomial.

Newton Forward Interpolating Polynomial

The **coefficients** of the Newton forward divided-difference form of the interpolating polynomial are along the **diagonal** in the table. **This polynomial is**

$$\begin{aligned} P_4(x) = {} & 0.7651977 - 0.4837057(x - 1.0) - 0.1087339(x - 1.0)(x - 1.3) \\ & + 0.0658784(x - 1.0)(x - 1.3)(x - 1.6) \\ & + 0.0018251(x - 1.0)(x - 1.3)(x - 1.6)(x - 1.9). \end{aligned}$$

0^{th} coefficient	1^{st} coefficient	2^{nd} coefficient	3^{rd} coefficient	4^{th} coefficient
(at $x_0 = 1.0$)	(uses x_0, x_1)	(uses x_0, x_1, x_2)	(uses x_0, x_1, x_2, x_3)	(uses x_0, x_1, x_2, x_3, x_4)

Nodes used: $x_0 = 1.0,\ x_1 = 1.3,\ x_2 = 1.6,\ x_3 = 1.9,\ x_4 = 2.2.$

Theorem (3.6)

Suppose that $f \in C^n[a, b]$ *and* $x_0, x_1, \ldots, x_n$ *are* ***distinct*** *numbers in* $[a, b]$.

Then ***there exists*** *a number* $\xi \in (a, b)$ *such that*

$$f[x_0, x_1, \ldots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

Interpretation:

The n–th divided difference of f at the distinct points $x_0, x_1, \ldots, x_n$ equals the n–th derivative of f evaluated at some point ξ in (a, b), divided by $n!$.

Newton Forward-Difference Formula

The **Newton forward–difference formula** is constructed by making use of the **forward difference** notation Δ introduced in Aitken's Δ^2 method. With this notation,

First divided difference 1

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{1}{h}\left(f(x_1) - f(x_0)\right) = \frac{1}{h}\Delta f(x_0)$$

Second divided difference 2

$$f[x_0, x_1, x_2] = \frac{1}{2h}\left[\frac{\Delta f(x_1) - \Delta f(x_0)}{h}\right] = \frac{1}{2h^2}\Delta^2 f(x_0)$$

General case k

$$f[x_0, x_1, \ldots, x_k] = \frac{1}{k!\,h^k}\Delta^k f(x_0)$$

Since $f[x_0] = f(x_0)$, Eq. (3.11) has the following form.

Newton's Forward-Difference Formula

$$P_n(x) = f(x_0) + \sum_{k=1}^{n} \binom{s}{k} \Delta^k f(x_0).$$

Definition (3.7)

Given the sequence $\{p_n\}_{n=0}^{\infty}$, define the **backward difference** ∇p_n (read *nabla* p_n) by

$$\nabla p_n = p_n - p_{n-1}, \qquad \text{for } n \geq 1.$$

Higher powers are defined recursively by

$$\nabla^k p_n = \nabla(\nabla^{k-1} p_n), \qquad \text{for } k \geq 2.$$

Table 3.12

x_i	$f(x_i)$	First divided differences	Second divided differences	Third divided differences	Fourth divided differences
1.0	0.7651977				
		−0.4837057			
1.3	0.6200860		−0.1087339		
		−0.5489460		0.0658784	
1.6	0.4554022		−0.0494433		
		−0.5786120		0.0680685	0.0018251
1.9	0.2818186		0.0118183		
		−0.5715210			
2.2	0.1103623				

Evaluate P_4 at $x = 1.1$

$$P_4(1.1) = P_4\left(1.0 + \frac{1}{3}(0.3)\right)$$

$$= 0.7651977 + \frac{1}{3}(0.3)(-0.4837057) + \frac{1}{3}\left(-\frac{2}{3}\right)(0.3)^2(-0.1087339)$$

$$+ \frac{1}{3}\left(-\frac{2}{3}\right)\left(-\frac{5}{3}\right)(0.3)^3(0.0658784)$$

$$+ \frac{1}{3}\left(-\frac{2}{3}\right)\left(-\frac{5}{3}\right)\left(-\frac{8}{3}\right)(0.3)^4(0.0018251)$$

$$= 0.7196460.$$

Evaluate P_4 at $x = 2.0$

$$P_4(2.0) = P_4\left(2.2 - \frac{2}{3}(0.3)\right)$$

$$= 0.1103623 - \frac{2}{3}(0.3)(-0.5715210) - \frac{2}{3}\left(\frac{1}{3}\right)(0.3)^2(0.0118183)$$

$$- \frac{2}{3}\left(\frac{1}{3}\right)\left(\frac{4}{3}\right)(0.3)^3(0.0680685) - \frac{2}{3}\left(\frac{1}{3}\right)\left(\frac{4}{3}\right)\left(\frac{7}{3}\right)(0.3)^4(0.0018251)$$

$$= 0.2238754.$$

Newton's Backward-Difference Formula

$$P_n(x) = f[x_n] + \sum_{k=1}^{n} (-1)^k \binom{-s}{k} \nabla^k f(x_n).$$

- Newton's forward and backward difference formulas are not appropriate for approximating $f(x)$ when x lies **near the center** of the table, since neither permits the highest-order difference to have x_0 close to x.
- A number of divided-difference formulas are available for this case.
- These methods are known as **centered-difference formulas**.

In this section

We consider only one centered-difference formula, **Stirling's method.**

Definition (Centered Difference: Stirling's Formula)

$$
\begin{aligned}
P_n(x) \quad = \quad & P_{2m+1}(x) = f[x_0] + \frac{sh}{2}\left(f[x_{-1},x_0] + f[x_0,x_1]\right) \\
& + s^2h^2\, f[x_{-1},x_0,x_1] + \frac{s(s^2-1)h^3}{2}\, f[x_{-2},x_{-1},x_0,x_1] \\
& + f[x_{-1},x_0,x_1,x_2] + \cdots \\
& + s^2(s^2-1)(s^2-4)\cdots(s^2-(m-1)^2)h^{2m}\, f[x_{-m},\ldots,x_m] \\
& + \frac{s(s^2-1)\cdots(s^2-m^2)h^{2m+1}}{2}\, f[x_{-m-1},\ldots,x_m] \\
& + f[x_{-m},\ldots,x_{m+1}],
\end{aligned}
$$

Notation

- x_0 — center of the table
- h — step size
- $s = \dfrac{x - x_0}{h}$ (dimensionless)
- $f[x_i,\ldots,x_j]$ divided difference

Note. If $n = 2m+1$ is odd, use the full formula above.

If $n = 2m$ is even, use the same formula but delete the last line (the term $+ f[x_{-m},\ldots,x_{m+1}]$).

If $n = 2m + 1$ is odd. If $n = 2m$ is even, we use the same formula but delete the last line. The entries used for this formula are underlined in Table 3.13. The entries used for Stirling's formula are underlined in this table.

Table 3.12

x	$f(x)$	First divided differences	Second divided differences	Third divided differences	Fourth divided differences
x_{-2}	$f[x_{-2}]$				
		$f[x_{-2}, x_{-1}]$			
x_{-1}	$f[x_{-1}]$		$f[x_{-2}, x_{-1}, x_0]$		
		$\underline{f[x_{-1}, x_0]}$		$\underline{f[x_{-2}, x_{-1}, x_0, x_1]}$	
x_0	$\underline{f[x_0]}$		$\underline{f[x_{-1}, x_0, x_1]}$		$\underline{f[x_{-2}, x_{-1}, x_0, x_1, x_2]}$
		$\underline{f[x_0, x_1]}$		$\underline{f[x_{-1}, x_0, x_1, x_2]}$	
x_1	$f[x_1]$		$f[x_0, x_1, x_2]$		
		$f[x_1, x_2]$			
x_2	$f[x_2]$				

Consider the table of data given in the previous examples. Use Stirling's formula to approximate $f(1.5)$ with $x_0 = 1.6$.

Table 3.14

x	$f(x)$	First divided differences	Second divided differences	Third divided differences	Fourth divided differences
1.0	0.7651977				
		−0.4837057			
1.3	0.6200860		−0.1087339		
		−0.5489460		0.0658784	
1.6	0.4554022		−0.0494433		0.0018251
		−0.5786120		0.0680685	
1.9	0.2818186		0.0118183		
		−0.5715210			
2.2	0.1103623				

Using Stirling's Formula

The formula, with $h = 0.3$, $x_0 = 1.6$, and $s = -\frac{1}{3}$, becomes

$$f(1.5) \approx P_4\left(1.6 + \left(-\frac{1}{3}\right)(0.3)\right)$$

$$= 0.4554022 + \left(-\frac{1}{3}\right)\left(\frac{0.3}{2}\right)((-0.5489460) + (-0.5786120))$$

$$+ \left(-\frac{1}{3}\right)^2 (0.3)^2 (-0.0494433)$$

$$+ \frac{1}{2}\left(-\frac{1}{3}\right)\left(\left(-\frac{1}{3}\right)^2 - 1\right)(0.3)^3 (0.0658784 + 0.0680685)$$

$$+ \left(-\frac{1}{3}\right)^2\left(\left(-\frac{1}{3}\right)^2 - 1\right)(0.3)^4 (0.0018251) = \underline{0.5118200}.$$

Therefore, $f(1.5) \approx 0.5118200$.

Definition (3.8)

Let $x_0, x_1, \ldots, x_n$ be $n+1$ distinct numbers in $[a, b]$ and for $i = 0, 1, \ldots, n$ let m_i be a nonnegative integer. Suppose that $f \in C^m[a, b]$, where $m = \max_{0 \le i \le n} m_i$.

- The **osculating polynomial** approximating f is the polynomial $P(x)$ of least degree such that

$$\frac{d^k P(x_i)}{dx^k} = \frac{d^k f(x_i)}{dx^k}, \quad \text{for each } i = 0, 1, \ldots, n \text{ and } k = 0, 1, \ldots, m_i.$$

- In previous sections, we introduced the interpolation $P(x)$ of a continuous function $f(x)$ at some given points

x_0	x_1	x_2	$\cdots$	x_n
$f(x_0)$	$f(x_1)$	$f(x_2)$	$\cdots$	$f(x_n)$

- In some applications, we want not only the interpolation function $P(x)$ to match with $f(x)$ at some points, but also the derivatives of $P(x)$ to match with the derivatives of $f(x)$ at those points. These interpolating polynomials are called **osculating polynomials**.

- In particular, we look for a polynomial $P(x)$ of the least degree such that $P(x)$ and $P'(x)$ agrees with f and its derivative f' at $n+1$ distinct points $x_0, x_1, \cdots, x_n$, i.e.,

$$\begin{aligned} P(x_i) &= f(x_i) \\ P'(x_i) &= f'(x_i) \end{aligned} \qquad i = 0, 1, \cdots, n.$$

- Recall the nth-degree **Lagrange interpolating polynomials**:

$$L_{n,k}(x) = \prod_{\substack{i \neq k \\ i=0}}^{n} \frac{(x - x_i)}{(x_k - x_i)} \qquad k = 0, 1, \cdots, n.$$

They satisfy $L_{n,k}(x_i) = \delta_{ik}$.

Construction of Hermite Interpolation (1/3)

Define $n + 1$ polynomials:

$$A_k(x) = \Big(1 - 2(x - x_k)L'_{n,k}(x_k)\Big)L^2_{n,k}(x) \qquad k = 0, 1, \cdots, n.$$

It can be verified that

- The degree of A_k is $2n + 1$.
- $A_k(x_i) = \delta_{ik}$.

Construction of Hermite Interpolation (2/3)

- Also, the derivative of $A_k(x)$ is

$$A_k'(x) = \Big(-2L_{n,k}'(x_k)\Big)L_{n,k}^2(x) + \Big(1-2(x-x_k)L_{n,k}'(x_k)\Big)2L_{n,k}(x)L_{n,k}'(x).$$

- Thus,

$$A_k'(x_k) = -2L_k'(x_k) + 2L_k'(x_i) = 0,$$

and

$$A_k'(x_i) = 0, \quad \text{for } i \neq k.$$

- In summary, we have

$$A_k(x_i) = \delta_{ik}, \quad A_k'(x_i) = 0.$$

Construction of Hermite Interpolation (3/3)

Define another $n+1$ polynomials:

$$B_k(x) = (x - x_k)L_{n,k}^2(x) \qquad k = 0, 1, \cdots, n.$$

It can be verified that

- The degree of $B_k(x)$ is $2n+1$.
- $B_k(x_i) = (x_i - x_k)L_{n,k}^2(x_i) = 0 \qquad k = 0, 1, \cdots, n.$

The derivative of $B_k(x)$ is

$$B_k'(x) = L_{n,k}^2(x) + 2(x - x_k)L_{n,k}(x)L_{n,k}'(x).$$

Thus, so that $B_k'(x_k) = 1$, and $B_k'(x_i) = 0$, for $i \neq k$. Hence,

$$B_k(x_i) = 0, \quad B_k'(x_i) = \delta_{ik}.$$

Hermite Interpolation

If $f \in C^1[a,b]$ and $x_0, x_1 \cdots, x_n \in [a,b]$ are distinct, the unique polynomial of least degree agreeing with f and f' at $x_0, x_1 \cdots, x_n$ is the Hermite polynomial of degree at most $2n+1$ given by

$$H_{2n+1}(x) = \sum_{k=0}^{n} f(x_k) A_k(x) + \sum_{k=0}^{n} f'(x_k) B_k(x).$$

where

$$\begin{aligned} A_k(x) &= \Big(1 - 2(x - x_k) L'_{n,k}(x_k)\Big) L^2_{n,k}(x), \\ B_k(x) &= (x - x_k) L^2_{n,k}(x), \end{aligned}$$

and $$L_{n,k}(x) = \prod_{\substack{i \neq k \\ i=0}}^{n} \frac{(x - x_i)}{(x_k - x_i)}.$$

Theorem (3.9)

If $f \in C^1[a, b]$ and $x_0, \ldots, x_n \in [a, b]$ are distinct, the unique polynomial of least degree agreeing with f and f' at $x_0, \ldots, x_n$ is the Hermite polynomial of degree at most $2n + 1$ given by

$$H_{2n+1}(x) = \sum_{j=0}^{n} f(x_j) H_{n,j}(x) + \sum_{j=0}^{n} f'(x_j) \hat{H}_{n,j}(x),$$

where, for $L_{n,j}(x)$ denoting the jth Lagrange coefficient polynomial of degree n, we have

$$H_{n,j}(x) = [1 - 2(x - x_j) L'_{n,j}(x_j)] L^2_{n,j}(x) \quad \text{and} \quad \hat{H}_{n,j}(x) = (x - x_j) L^2_{n,j}(x).$$

Moreover, if $f \in C^{2n+2}[a, b]$, then

$$f(x) = H_{2n+1}(x) + \frac{(x - x_0)^2 \ldots (x - x_n)^2}{(2n + 2)!} f^{(2n+2)}(\xi(x)),$$

for some (generally unknown) $\xi(x)$ in the interval (a, b).

Determine the Hermite polynomial that interpolates the data below

x	1	2
$f(x)$	1	-3
$f'(x)$	-1	2

Solution(1/2)

- First compute the Lagrange polynomials of the point $x_0 = 1$ and $x_1 = 2$.

$$L_0(x) = -(x-2), \quad L_1(x) = x - 1$$

- Next, we compute third-order polynomials ($n = 1,\ 2n+1 = 3$).

$$\begin{aligned}
A_0(x) &= (1 - 2(x-1)(-1))(x-2)^2 = (2x-1)(x-2)^2 \\
A_1(x) &= (1 - 2(x-2))(x-1)^2 = -(2x-5)(x-1)^2 \\
B_0(x) &= (x-1)(x-2)^2, \\
B_1(x) &= (x-2)(x-1)^2
\end{aligned}$$

Solution (2/2)

Then, the Hermite interpolating polynomial is

$$\begin{aligned} H_3(x) &= f(x_0)A_0(x) + f(x_1)A_1(x) + f'(x_0)B_0(x) + f'(x_1)B_1(x) \\ &= 1 \cdot (2x-1)(x-2)^2 + (-3) \cdot -(2x-5)(x-1)^2 \\ &\quad +(-1) \cdot (x-1)(x-2)^2 + 2 \cdot (x-2)(x-1)^2 \\ &= 9x^3 - 39x^2 + 50x - 19. \end{aligned}$$

Problem

Use the Hermite polynomial that agrees with the data in the table to approximate $f(1.5)$.

Data Table

k	x_k	$f(x_k)$	$f'(x_k)$
0	1.3	0.6200860	−0.5220232
1	1.6	0.4554022	−0.5698959
2	1.9	0.2818186	−0.5811571

Hermite Interpolation: Example (1/2)

Step 1: Compute the Lagrange basis polynomials and their derivatives.

Problem

Use the Hermite polynomial that agrees with the data in the table to approximate $f(1.5)$.

Data Table

k	x_k	$f(x_k)$	$f'(x_k)$
0	1.3	0.6200860	−0.5220232
1	1.6	0.4554022	−0.5698959
2	1.9	0.2818186	−0.5811571

Lagrange Basis Polynomials $L_{2,i}(x)$ and Their Derivatives $L'_{2,i}(x)$

	$L_{2,i}(x)$	$L'_{2,i}(x)$
$i = 0$	$L_{2,0}(x) = \frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9}$	$L'_{2,0}(x) = \frac{100}{9}x - \frac{175}{9}$
$i = 1$	$L_{2,1}(x) = -\frac{100}{9}x^2 + \frac{320}{9}x - \frac{247}{9}$	$L'_{2,1}(x) = -\frac{200}{9}x + \frac{320}{9}$
$i = 2$	$L_{2,2}(x) = \frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9}$	$L'_{2,2}(x) = \frac{100}{9}x - \frac{145}{9}$

These Lagrange polynomials and derivatives will be used to build the Hermite basis functions on the next slide.

Hermite Interpolation: Example (2/2)

Step 2: Build the Hermite basis functions and evaluate at $x = 1.5$.

Hermite basis functions

$$H_{2,i}(x) = [1 - 2(x - x_i)L'_{2,i}(x_i)]\,L^2_{2,i}(x)$$

$$\hat{H}_{2,i}(x) = (x - x_i)L^2_{2,i}(x)$$

$$H_5(x) = \sum_{i=0}^{2} [f(x_i)H_{2,i}(x) + f'(x_i)\hat{H}_{2,i}(x)]$$

Target: $x = 1.5$

Computed basis terms

$$H_{2,0}(x) = (10x - 12)\left(\frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9}\right)^2$$

$$H_{2,1}(x) = \left(-\frac{100}{9}x^2 + \frac{320}{9}x - \frac{247}{9}\right)^2$$

$$H_{2,2}(x) = 10(2 - x)\left(\frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9}\right)^2$$

$$\hat{H}_{2,0}(x) = (x - 1.3)\left(\frac{50}{9}x^2 - \frac{175}{9}x + \frac{152}{9}\right)^2$$

$$\hat{H}_{2,1}(x) = (x - 1.6)\left(-\frac{100}{9}x^2 + \frac{320}{9}x - \frac{247}{9}\right)^2$$

$$\hat{H}_{2,2}(x) = (x - 1.9)\left(\frac{50}{9}x^2 - \frac{145}{9}x + \frac{104}{9}\right)^2$$

Evaluate at $x = 1.5$

$$\begin{aligned} H_5(1.5) &= 0.6200860\left(\frac{4}{27}\right) + 0.4554022\left(\frac{64}{81}\right) + 0.2818186\left(\frac{5}{81}\right) \\ &\quad - 0.5220232\left(\frac{4}{405}\right) - 0.5698959\left(-\frac{32}{405}\right) - 0.5811571\left(-\frac{2}{405}\right) \\ &= 0.5118277 \end{aligned}$$

$$\rightarrow \quad H_5(1.5) \approx 0.5118277$$

Hermite Interpolation via Divided Differences

1 Motivation

- There is an alternative approach to construct Hermite interpolation using Newton's divided differences.
- Since each data point x_i is associated with two values $f(x_i)$ and $f'(x_i)$.

2 Repeated nodes

$$z_{2i} = z_{2i+1} = x_i, \qquad i = 0, 1, \cdots, n.$$

- We construct the divided-difference table using $z_0, z_1, \cdots, z_{2n+1}$.

3 Key modification

- Since $z_{2i} = z_{2i+1} = x_i$, the standard first divided difference

$$f[z_{2i}, z_{2i+1}] = \frac{f[z_{2i+1}] - f[z_{2i}]}{z_{2i+1} - z_{2i}}$$

cannot be used.

- The appropriate replacement is

$$\boxed{f[z_{2i}, z_{2i+1}] = f'(x_i).}$$

Hermite Interpolation: Repeated-Node Table

1 Original data

Given the nodes and function values with derivatives:

x_0	x_1	x_2	$\cdots$	x_n
$f(x_0)$	$f(x_1)$	$f(x_2)$	$\cdots$	$f(x_n)$
$f'(x_0)$	$f'(x_1)$	$f'(x_2)$	$\cdots$	$f'(x_n)$

2 Duplicate each node

For each node x_i, create two copies:

$$z_{2i} = z_{2i+1} = x_i, \quad i = 0, 1, \ldots, n.$$

x_0 (twice)		x_1 (twice)		x_2 (twice)		$\cdots$	x_n (twice)	
x_0	x_0	x_1	x_1	x_2	x_2	$\cdots$	x_n	x_n
z_0	z_1	z_2	z_3	z_4	z_5	$\cdots$	z_{2n}	z_{2n+1}
$f(z_0)$	$f(z_1)$	$f(z_2)$	$f(z_3)$	$f(z_4)$	$f(z_5)$	$\cdots$	$f(z_{2n})$	$f(z_{2n+1})$
$f(x_0)$	$f(x_0)$	$f(x_1)$	$f(x_1)$	$f(x_2)$	$f(x_2)$	$\cdots$	$f(x_n)$	$f(x_n)$

3 Special first divided differences

For each repeated pair, the usual quotient $\dfrac{f[z_{2i+1}] - f[z_{2i}]}{z_{2i+1} - z_{2i}}$ is undefined because $z_{2i+1} - z_{2i} = 0$.

In Hermite interpolation, we define instead

$$f[z_{2i}, z_{2i+1}] = f'(x_i), \quad i = 0, 1, \ldots, n.$$

Example: $f[z_0, z_1] = f'(x_0)$.

All other divided differences are computed in the usual way.

Divided-Difference Table for Hermite Polynomial

z	$f(z)$	First divided differences	Second divided differences
$z_0 = x_0$	$f[z_0] = f(x_0)$		
		$f[z_0, z_1] = f'(x_0)$	
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2] = \dfrac{f[z_1, z_2] - f[z_0, z_1]}{z_2 - z_0}$
		$f[z_1, z_2] = \dfrac{f[z_2] - f[z_1]}{z_2 - z_1}$	
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3] = \dfrac{f[z_2, z_3] - f[z_1, z_2]}{z_3 - z_1}$
		$f[z_2, z_3] = f'(x_1)$	
$z_3 = x_1$	$f[z_3] = f(x_1)$		$f[z_2, z_3, z_4] = \dfrac{f[z_3, z_4] - f[z_2, z_3]}{z_4 - z_2}$
		$f[z_3, z_4] = \dfrac{f[z_4] - f[z_3]}{z_4 - z_3}$	
$z_4 = x_2$	$f[z_4] = f(x_2)$		$f[z_3, z_4, z_5] = \dfrac{f[z_4, z_5] - f[z_3, z_4]}{z_5 - z_3}$
		$f[z_4, z_5] = f'(x_2)$	
$z_5 = x_2$	$f[z_5] = f(x_2)$		

$$H_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, z_1, \cdots, z_k](x - z_0)(x - z_1)\cdots(x - z_{k-1}).$$

Divided-Difference Table for Hermite Polynomial

Repeated nodes	Function values	First divided differences	Second divided differences
$z_0 = x_0$	$f[z_0] = f(x_0)$	$f[z_0, z_1] = f'(x_0)$	
$z_1 = x_0$	$f[z_1] = f(x_0)$	$f[z_1, z_2] = \dfrac{f[z_2] - f[z_1]}{z_2 - z_1}$	$f[z_0, z_1, z_2] = \dfrac{f[z_1, z_2] - f[z_0, z_1]}{z_2 - z_0}$
$z_2 = x_1$	$f[z_2] = f(x_1)$	$f[z_2, z_3] = f'(x_1)$	$f[z_1, z_2, z_3] = \dfrac{f[z_2, z_3] - f[z_1, z_2]}{z_3 - z_1}$
$z_3 = x_1$	$f[z_3] = f(x_1)$	$f[z_3, z_4] = \dfrac{f[z_4] - f[z_3]}{z_4 - z_3}$	$f[z_2, z_3, z_4] = \dfrac{f[z_3, z_4] - f[z_2, z_3]}{z_4 - z_2}$
$z_4 = x_2$	$f[z_4] = f(x_2)$	$f[z_4, z_5] = f'(x_2)$	$f[z_3, z_4, z_5] = \dfrac{f[z_4, z_5] - f[z_3, z_4]}{z_5 - z_3}$
$z_5 = x_2$	$f[z_5] = f(x_2)$		

Newton form of the Hermite interpolating polynomial

$$H_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, z_1, \ldots, z_k](x - z_0)(x - z_1)\cdots(x - z_{k-1}).$$

Determine the Hermite polynomial that interpolates the data below

x	1	2
$f(x)$	1	-3
$f'(x)$	-1	2

using Newton's divided differences table.

Solution

The divided difference table is

$$
\begin{array}{llllll}
z_0 = 1 & f[z_0] = 1 & f[z_0, z_1] = -1 & f[z_0, z_1, z_2] = -3 & f[z_0, z_1, z_2, z_3] = 9 \\
z_1 = 1 & f[z_1] = 1 & f[z_1, z_2] = -4 & f[z_1, z_2, z_3] = 6 & \\
z_2 = 2 & f[z_2] = -3 & f[z_2, z_3] = 2 & & \\
z_3 = 2 & f[z_3] = -3 & & &
\end{array}
$$

Then the 3rd-degree Hermite interpolating polynomial is

$$
\begin{aligned}
H_3(x) &= 1 - (x-1) - 3(x-1)^2 + 9(x-1)^2(x-2) \\
&= 9x^3 - 39x^2 + 50x - 19.
\end{aligned}
$$

For the error bound of the Hermite interpolation, we have the following result.

Theorem 12 (Hermite Interpolation Error Theorem).

Suppose $x_0, x_1, \cdots, x_n$ *are distinct numbers in the interval* $[a, b]$ *and* $f \in C^{2n+2}[a, b]$. *Then, for each* $x \in [a, b]$, *there exists a number* ξ *in* (a, b) *(generally unknown), such that*

$$f(x) = H_{2n+1}(x) + \frac{f^{(2n+2)}(\xi)}{(2n+2)!}(x - x_0)^2(x - x_1)^2 \cdots (x - x_n)^2,$$

where $P_{2n+1}(x)$ *is the Hermite interpolating polynomial of* $f(x)$.

$$P_n(x) = f[x_0] + \sum_{k=1}^{n} f[x_0, x_1, \ldots, x_k](x - x_0) \cdots (x - x_{k-1}).$$

Suppose that the distinct numbers $x_0, x_1, \ldots, x_n$ are given together with the values of f and f' at these numbers. Define a new sequence $z_0, z_1, \ldots, z_{2n+1}$ by

$$z_{2i} = z_{2i+1} = x_i, \quad \text{for each } i = 0, 1, \ldots, n,$$

$$f[z_{2i}, z_{2i+1}] = f'(z_{2i}) = f'(x_i).$$

$$H_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, \ldots, z_k](x - z_0)(x - z_1) \cdots (x - z_{k-1}).$$

z	$f(z)$	First divided differences	Second divided differences
$z_0 = x_0$	$f[z_0] = f(x_0)$		
		$f[z_0, z_1] = f'(x_0)$	
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2] = \dfrac{f[z_1, z_2] - f[z_0, z_1]}{z_2 - z_0}$
		$f[z_1, z_2] = \dfrac{f[z_2] - f[z_1]}{z_2 - z_1}$	
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3] = \dfrac{f[z_2, z_3] - f[z_1, z_2]}{z_3 - z_1}$
		$f[z_2, z_3] = f'(x_1)$	
$z_3 = x_1$	$f[z_3] = f(x_1)$		$f[z_2, z_3, z_4] = \dfrac{f[z_3, z_4] - f[z_2, z_3]}{z_4 - z_2}$
		$f[z_3, z_4] = \dfrac{f[z_4] - f[z_3]}{z_4 - z_3}$	
$z_4 = x_2$	$f[z_4] = f(x_2)$		$f[z_3, z_4, z_5] = \dfrac{f[z_4, z_5] - f[z_3, z_4]}{z_5 - z_3}$
		$f[z_4, z_5] = f'(x_2)$	
$z_5 = x_2$	$f[z_5] = f(x_2)$		

Hermite Interpolation via Divided Differences: Example

Use the data from Example 1 to determine the Hermite polynomial approximation at $x = 1.5$.

$$\frac{0.4554022 - 0.6200860}{1.6 - 1.3} = -0.5489460.$$

z_i	$f[z_i]$	$f[z_i, z_{i+1}]$	$f[z_i, z_{i+1}, z_{i+2}]$	$f[z_i, z_{i+1}, z_{i+2}, z_{i+3}]$	$f[z_i, z_{i+1}, z_{i+2}, z_{i+3}, z_{i+4}]$	$f[z_i, z_{i+1}, z_{i+2}, z_{i+3}, z_{i+4}, z_{i+5}]$
1.3	0.6200860	−0.5220232				
			−0.0897427			
1.3	0.6200860	−0.5489460		0.0663657		
			−0.0698330		0.0026663	
1.6	0.4554022	−0.5698959		0.0679655		−0.0027738
			−0.0290537		0.0010020	
1.6	0.4554022	−0.5786120		0.0685667		
			−0.0084837			
1.9	0.2818186	−0.5811571				
1.9	0.2818186					

$$\begin{aligned}
H_5(1.5) &= f[1.3] + f'[1.3](1.5 - 1.3) + f[1.3, 1.3, 1.6](1.5 - 1.3)^2 \\
&\quad + f[1.3, 1.3, 1.6, 1.6](1.5 - 1.3)^2(1.5 - 1.6) \\
&\quad + f[1.3, 1.3, 1.6, 1.6, 1.9](1.5 - 1.3)^2(1.5 - 1.6)^2 \\
&\quad + f[1.3, 1.3, 1.6, 1.6, 1.9, 1.9](1.5 - 1.3)^2(1.5 - 1.6)^2(1.5 - 1.9) \\
&= 0.6200860 + (-0.5220232)(0.2) + (-0.0897427)(0.2)^2 \\
&\quad + 0.0663657(0.2)^2(-0.1) + 0.0026663(0.2)^2(-0.1)^2 \\
&\quad + (-0.0027738)(0.2)^2(-0.1)^2(-0.4) \\
&= 0.5118277.
\end{aligned}$$

$$H_5(1.5) \approx 0.5118277$$

Algorithm 3.3: HERMITE INTERPOLATION

To obtain the coefficients of the Hermite interpolating polynomial $H(x)$ on the $(n+1)$ distinct numbers $x_0, \ldots, x_n$ for the function f:

INPUT numbers $x_0, x_1, \ldots, x_n$; values $f(x_0), \ldots, f(x_n)$ and $f'(x_0), \ldots, f'(x_n)$.
OUTPUT the numbers $Q_{0,0}, Q_{1,1}, \ldots, Q_{2n+1,2n+1}$ where

$$H(x) = Q_{0,0} + Q_{1,1}(x - x_0) + Q_{2,2}(x - x_0)^2 + Q_{3,3}(x - x_0)^2(x - x_1) + Q_{4,4}(x - x_0)^2(x - x_1)^2 + \cdots + Q_{2n+1,2n+1}(x - x_0)^2(x - x_1)^2 \cdots (x - x_{n-1})^2(x - x_n).$$

Step 1 For $i = 0, 1, \ldots, n$ do Steps 2 and 3.

Step 2 Set $z_{2i} = x_i$;
$z_{2i+1} = x_i$;
$Q_{2i,0} = f(x_i)$;
$Q_{2i+1,0} = f(x_i)$;
$Q_{2i+1,1} = f'(x_i)$.

Step 3 If $i \neq 0$ then set

$$Q_{2i,1} = \frac{Q_{2i,0} - Q_{2i-1,0}}{z_{2i} - z_{2i-1}}.$$

Step 4 For $i = 2, 3, \ldots, 2n+1$

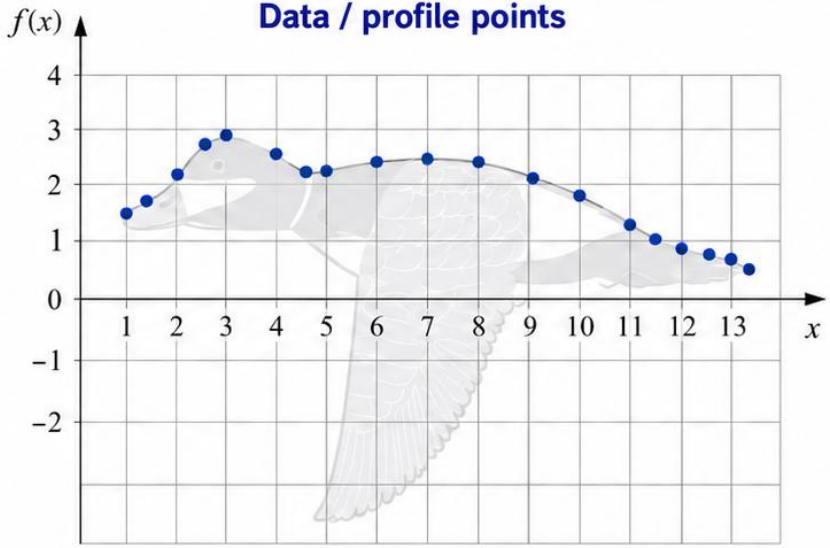
for $j = 2, 3, \ldots, i$ set $Q_{i,j} = \dfrac{Q_{i,j-1} - Q_{i-1,j-1}}{z_i - z_{i-j}}$.

Step 5 OUTPUT $(Q_{0,0}, Q_{1,1}, \ldots, Q_{2n+1,2n+1})$;
STOP

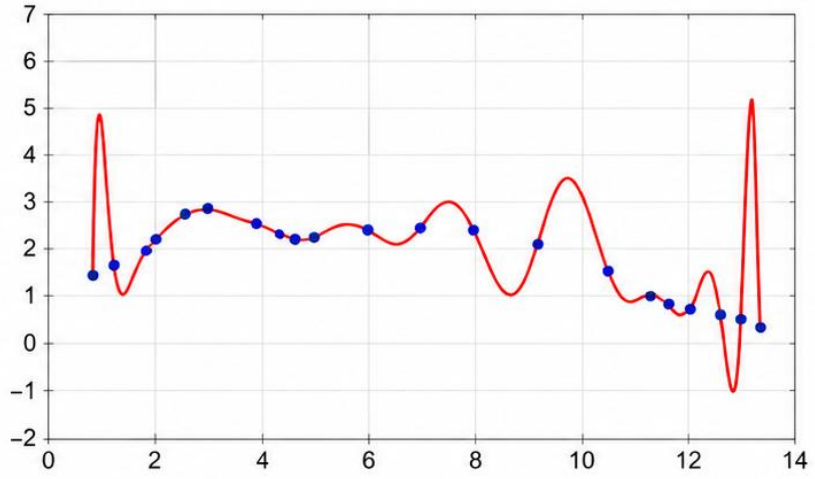
Cubic Spline Interpolation

- In previous sections, we approximated functions on closed intervals using a **single polynomial**.
- However, high-degree polynomials can **oscillate** erratically: a small local fluctuation may produce large variations over the entire interval.
- The figure below shows a 20th-degree Lagrange interpolant for the back of a duck.
  The resulting curve does not accurately reflect the true profile of the back.

Data / profile points



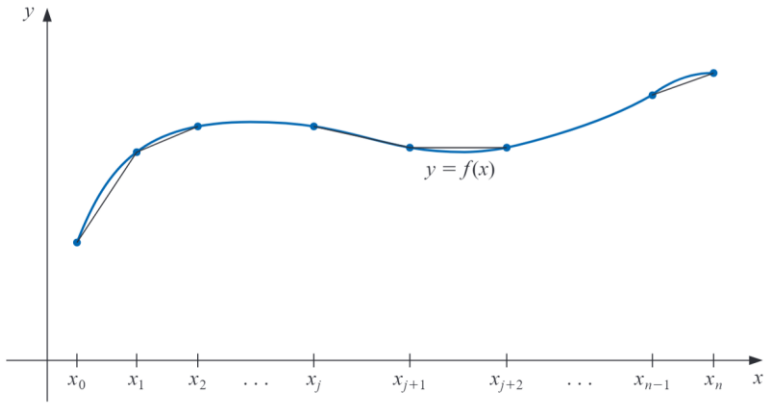
20th-degree polynomial interpolant



- In this section, we introduce the interpolation using **piecewise polynomials**. This will effectively prevent the oscillation.
- The simplest piecewise-polynomial approximation is **piecewise-linear interpolation**. Consider a set of data points:

x_0	x_1	x_2	$\cdots$	x_n
$f(x_0)$	$f(x_1)$	$f(x_2)$	$\cdots$	$f(x_n)$

- The following is a piecewise linear polynomial interpolation of a smooth curve.

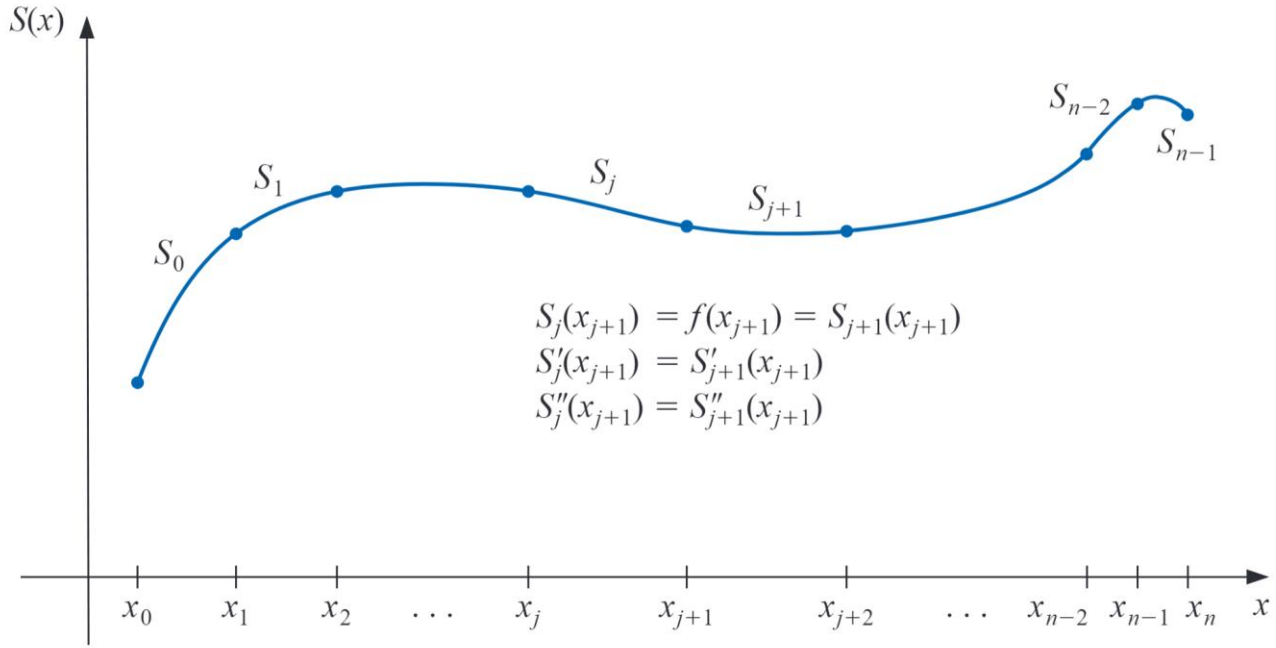


- The **disadvantage** of using piecewise linear approximation is that the approximation function is not ”smooth”, i.e., it is not differentiable at nodes x_i.
- Often it is clear from physical conditions that **smoothness is required**, so the approximating function must be continuously differentiable.
- Using Hermite polynomials could be a choice, however this requires the known of the derivative values at nodes.

The goal in this section is to develop a new interpolation so that

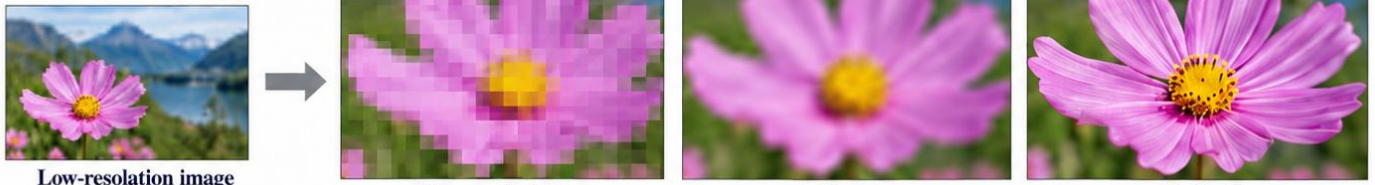
- it uses piecewise polynomials (to avoid oscillation).
- it is continuously differentiable over the entire domain (to ensure the smoothness).
- it requires no specific derivative information of the original function, except perhaps at the two endpoints of the interval (minimum information from original function).

- The most common piecewise-polynomial approximation is called **cubic spline interpolation**.
- The interpolation uses piecewise cubic polynomials, and globally second-order differentiable ($S \in C^2([x_0, x_n])$).



Examples of Bicubic Interpolation

A Photo Enlargement



Low-resolation image (64 × 48) | Nearest Neighbor (enlarged 4×) | Bilinear (enlarged 4×) | Bicubic (enlarged 4×)

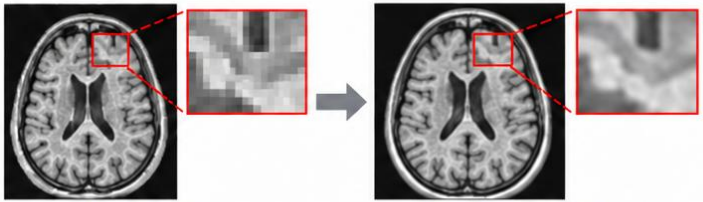
Bicubic interpolation is widely used for enlarging photographs.

B Image Rotation

Original image (400 × 300) | Bicubic resampling (rotated 30°)

Useful when rotating, aligning, or transforming images.

C Scientific / Medical Resampling



Original scan (256 × 256) | Bicubic resampling (enlarged 2×)

Helps when displaying scans at different resolutions.

D Key idea

Bicubic interpolation estimates new pixel values using nearby pixels and cubic polynomials, producing smoother visual results.

Remark: Cubic interpolation is 1D.
For 2D images, the correct term is bicubic interpolation.
So image resizing examples like the sunflower use bicubic interpolation.

Definition (3.10)

Given a function f defined on $[a, b]$ and a set of nodes $a = x_0 < x_1 < \cdots < x_n = b$, a **cubic spline interpolant** S for f is a function that satisfies the following conditions:

(a) $S(x)$ is a cubic polynomial, denoted $S_j(x)$, on the subinterval $[x_j, x_{j+1}]$ for each $j = 0, 1, \ldots, n-1$;

(b) $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$ for each $j = 0, 1, \ldots, n-1$;

(c) $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$; (*Implied by* **(b)**.)

(d) $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$;

(e) $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$;

(f) One of the following sets of boundary conditions is satisfied:

- **(i)** $S''(x_0) = S''(x_n) = 0$ **(natural** *(or free)* **boundary)**;
- **(ii)** $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$ **(clamped boundary)**.

Theorem (3.11)

If f is defined at $a = x_0 < x_1 < \cdots < x_n = b$, then f has a unique natural spline interpolant S on the nodes $x_0, x_1, \ldots, x_n$; that is, a spline interpolant that satisfies the natural boundary conditions $S''(a) = 0$ and $S''(b) = 0$.

Algorithm 3.4: NATURAL CUBIC SPLINE

To construct the cubic spline interpolant S for the function f, defined at the numbers $x_0 < x_1 < \cdots < x_n$, satisfying $S''(x_0) = S''(x_n) = 0$:
INPUT n; $x_0, x_1, \ldots, x_n$; $a_0 = f(x_0), a_1 = f(x_1), \ldots, a_n = f(x_n)$.
OUTPUT a_j, b_j, c_j, d_j for $j = 0, 1, \ldots, n-1$.
(*Note:* $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$ *for* $x_j \le x \le x_{j+1}$.)

Step 1 For $i = 0, 1, \ldots, n-1$ set $h_i = x_{i+1} - x_i$.

Step 2 For $i = 1, 2, \ldots, n-1$ set $\alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1})$.

Step 3 Set $l_0 = 1$; $\mu_0 = 0$; $z_0 = 0$. (*Steps 3-5, & part of 6 solve tridiagonal linear system by method in Algorithm 6.7.*)

Step 4 For $i = 1, 2, \ldots, n-1$
set $l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1}$;
$\mu_i = h_i/l_i$; $z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i$.

Step 5 Set $l_n = 1$; $z_n = 0$; $c_n = 0$.

Step 6 For $j = n-1, n-2, \ldots, 0$
set $c_j = z_j - \mu_j c_{j+1}$; $b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3$;
$b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3$;

Step 7 OUTPUT $(a_j, b_j, c_j, d_j$ for $j = 0, 1, \ldots, n-1)$;
STOP.

Theorem (3.12)

If f is defined at $a = x_0 < x_1 < \cdots < x_n = b$ and differentiable at a and b, then f has a unique clamped spline interpolant S on the nodes $x_0, x_1, \ldots, x_n$; that is, a spline interpolant that satisfies the clamped boundary conditions $S'(a) = f'(a)$ and $S'(b) = f'(b)$.

Algorithm 3.5: CLAMPED CUBIC SPLINE

To construct the cubic spline interpolant S for the function f defined at the numbers $x_0 < x_1 < \cdots < x_n$, satisfying $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$:

INPUT n; $x_0, x_1, \ldots, x_n$; $a_0 = f(x_0), a_1 = f(x_1), \ldots, a_n = f(x_n)$; $FPO = f'(x_0)$; $FPN = f'(x_n)$.

OUTPUT a_j, b_j, c_j, d_j for $j = 0, 1, \ldots, n-1$.

(*Note:* $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$ *for* $x_j \le x \le x_{j+1}$.)

Step 1 For $i = 0, 1, \ldots, n-1$ set $h_i = x_{i+1} - x_i$.

Step 2 Set $\alpha_0 = 3(a_1 - a_0)/h_0 - 3FPO$; $\alpha_n = 3FPN - 3(a_n - a_{n-1})/h_{n-1}$.

$\alpha_n = 3FPN - 3(a_n - a_{n-1})/h_{n-1}$.

Step 3 For $i = 1, 2, \ldots, n-1$

set $\alpha_i = \dfrac{3}{h_i}(a_{i+1} - a_i) - \dfrac{3}{h_{i-1}}(a_i - a_{i-1})$.

Step 4 Set $l_0 = 2h_0$; $\mu_0 = 0.5$; $_0 = \alpha_0/l_0$. (*Steps 4-6, & part of 7 solve a tridiagonal linear system by method in Algorithm 6.7.*)

Step 5 For $i = 1, 2, \ldots, n-1$

set $l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1}$; $\mu_i = h_i/l_i$; $z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i$.

Step 6 Set $l_n = h_{n-1}(2 - \mu_{n-1})$; $z_n = (\alpha_n - h_{n-1}z_{n-1})/l_n$; $c_n = z_n$.

Step 7 For $j = n-1, n-2, \ldots, 0$

set $c_j = z_j - \mu_j c_{j+1}$; $b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3$; $d_j = (c_{j+1} - c_j)/(3h_j)$.

Step 8 OUTPUT $(a_j, b_j, c_j, d_j$ for $j = 0, 1, \ldots, n-1)$;

STOP.

Construction of a natural cubic spline

- Assume the following $n+1$ distinct points

x_0	x_1	x_2	$\cdots$	x_n
$f(x_0)$	$f(x_1)$	$f(x_2)$	$\cdots$	$f(x_n)$

  subdivide the interval $[x_0, x_n]$ into n subintervals: $I_j = [x_j, x_{j+1}]$, $j = 0, 1, \cdots, n-1$.

- The cubic spline $S(x)$ restricted on the interval I_j is a cubic polynomial $S_j(x)$ for each $0 \le j \le n-1$:

$$S(x)|_{I_j} = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.$$

- There are $4n$ constants a_j, b_j, c_j, and d_j to be determined.

- On the interval $[x_j, x_{j+1}]$, denote the length $h_j = x_{j+1} - x_j$. We note that

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.$$

- It is easy to see that

$$a_j = f(x_j), \quad j = 0, 1, \cdots, n-1. \tag{3.1}$$

- If we also let $a_n = f(x_n)$, then by some calculations we can represent b_j and d_j in terms of c_j for $0 \le j \le n-1$:

$$\begin{aligned} b_j &= \frac{1}{h_j}(a_{j+1} - a_j) - \frac{h_j}{3}(2c_j + c_{j+1}), \\ d_j &= \frac{1}{3h_j}(c_{j+1} - c_j). \end{aligned} \tag{3.2}$$

  Here, we also use the definition $c_n = S_n''(x_n)/2$.

- We obtain a linear system for c_j, $1 \le j \le n-1$:

$$h_{j-1}c_{j-1} + 2(h_{j-1}+h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1}-a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}).$$

- Write it as a linear system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ h_0 & 2(h_0+h_1) & h_1 & & \\ 0 & h_1 & 2(h_1+h_2) & h_2 & \\ \vdots & \ddots & \ddots & \ddots & 0 \\ & & h_{n-2} & 2(h_{n-2}+h_{n-1}) & h_{n-1} \\ 0 & \cdots & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} 0 \\ \frac{3}{h_1}(a_2-a_1) - \frac{3}{h_0}(a_1-a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n-a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1}-a_{n-2}) \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}$$

- The matrix A is strictly diagonally dominant, that is,

$$|a_{ii}| > \sum_{j=1}^{n} |a_{ij}|, \quad \text{for all } i = 1, 2, \cdots, n.$$

- The matrix A is nonsingular and the linear system $A\mathbf{x} = \mathbf{b}$ has a unique solution(more in Chapter 6).

- To solve the linear system $A\mathbf{x} = \mathbf{b}$ with Matlab, we can use the command “backslash”:

$$\mathbf{x} = A \setminus \mathbf{b}.$$

Matlab File for Natural Spline Coefficient

```
function [a,b,c,d] = natural_spline_coef(dataX,dataY)
%% Cubic Spline (Natual Boundary) Interpolation
%  On each sub-interval, generate four coefficients aj, bj, cj, and dj
%       Sj(x) = aj + bj(x-xj) + cj(x-xj)^2 + dj(x-xj)^3.
%
%  Inputs: dataX dataY are row vectors of same dimension.
%  Output: a is the collum vector that stores {aj} for j=1,2,...,n+1
%          b is the collum vector that stores {bj} for j=1,2,...,n+1
%          c is the collum vector that stores {cj} for j=1,2,...,n+1
%          d is the collum vector that stores {dj} for j=1,2,...,n+1

%% Initialize h
n = length(dataX) - 1;
h = zeros(1,n);
for j = 1:n
    h(j) = dataX(j+1) - dataX(j);
end
%% Prepare for Matrix A
A = zeros(n+1,n+1);
A(1,1) = 1; A(n+1,n+1) = 1;
for j = 2:n
    A(j,j-1) = h(j-1);
    A(j,j+1) = h(j);
    A(j,j) = 2*(h(j-1)+h(j));
end
%% Prepare for vector bb
bb = zeros(n+1,1);
for j = 2:n
    bb(j) = 3/h(j)*(dataY(j+1)-dataY(j)) - 3/h(j-1)*(dataY(j)-dataY(j-1));
end
%% Solve A*c = bb
c = A\bb;
%% Find a,b,d.
a = reshape(dataY(1:n+1),n+1,1);
b = zeros(n,1);
d = zeros(n,1);
for j = 1:n
    b(j) = 1/h(j)*(a(j+1)-a(j)) - h(j)/3*(2*c(j)+c(j+1));
    d(j) = (c(j+1) - c(j))/(3*h(j));
end
%% Remove the last entries of a and c
a(n+1) = [];
c(n+1) = [];
```

Matlab File for Natural Spline Interpolation

```
function y = natural_spline(dataX,dataY,x)
%% Cubic Spline (Natual Boundary) Interpolation
%  On each sub-interval, generate four coefficients aj, bj, cj, and dj
%       Sj(x) = aj + bj(x-xj) + cj(x-xj)^2 + dj(x-xj)^3.
%
%  Inputs: dataX dataY are row vectors of same dimension.
%          x is the query point(s).
%  Output: y is the value of spline interpolation y=S(x) at query points x

%% Generate Spline Coefficient [ai,bi,ci,di] on each interval.
[a,b,c,d] = natural_spline_coef(dataX,dataY);

%% Evaluate the query points
y = zeros(size(x));
for n = 1:length(x)
    for j = 1:length(dataX)-1
        if dataX(j) <= x(n) && dataX(j+1) >= x(n)
            k = j;
            break
        end
    end
    xk = dataX(k);
    y(n) = a(k) + b(k)*(x(n)-xk) + c(k)*(x(n)-xk)^2 + d(k)*(x(n)-xk)^3;
end
```

At the beginning of Chapter 3, we gave some Taylor polynomial to approximate the exponential function $f(x) = e^x$. Use the data points

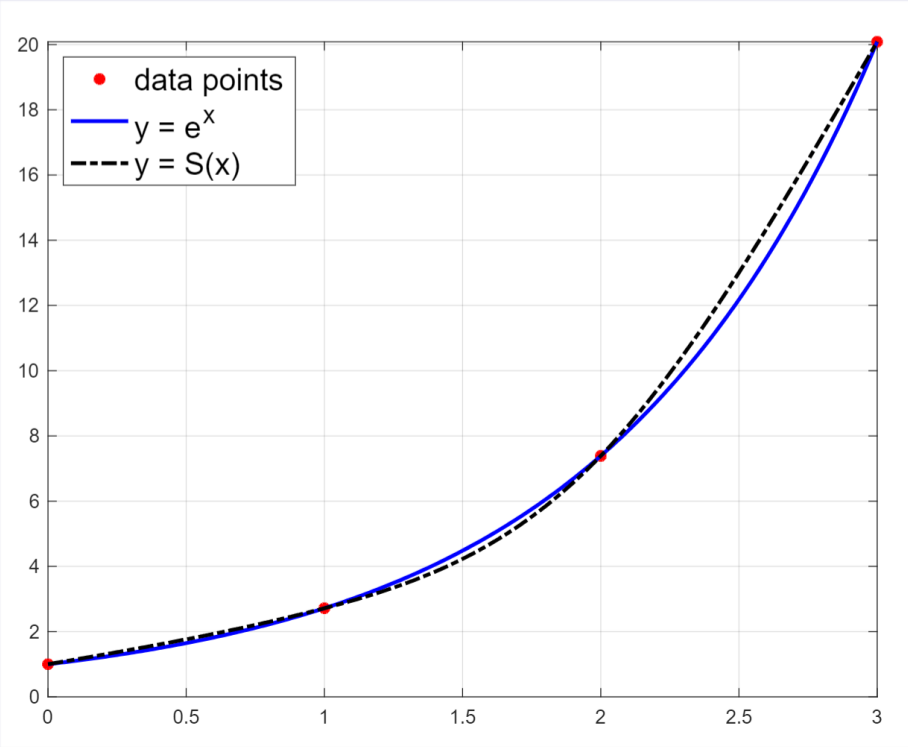
x	0	1	2	3
$f(x)$	1	e	e^2	e^3

to form a natural cubic spline $S(x)$ that approximates $f(x) = e^x$.

The MATLAB driver file for this example

```
% ex 3.4.1
clc
dataX = [0,1,2,3];
dataY = [exp(0),exp(1),exp(2),exp(3)];

%% generate coefficient of natural spline
[a,b,c,d] = natural_spline_coef(dataX,dataY);
disp('    a         b         c         d')
disp('-----------------------------------------')
disp([a,b,c,d])

%% query point x
x = 0:0.01:3;
y = natural_spline(dataX,dataY,x);

%% plot
figure(1)
plot(dataX,dataY,'r*','linewidth',2)
hold on
plot(x,exp(x),'b-','linewidth',2)
plot(x,y,'k-.','linewidth',2)
lgd = legend('data points','y = e^x','y = S(x)');
hold off
grid on
lgd.FontSize = 16;
lgd.Location = 'NorthWest';
```

$$S(x) = \begin{cases} 1.0000 + 1.4660x + 0.2523x^3, & \text{on } [0,1], \\ 2.7183 + 2.2229(x-1) + 0.7569(x-1)^2 + 1.69107(x-1)^3, & \text{on } [1,2], \\ 7.3891 + 8.8098(x-2) + 5.8301(x-2)^2 - 1.9434(x-2)^3, & \text{on } [2,3]. \end{cases}$$



Clamped Splines

- The **clamped spline** is another type of cubic splines that use different boundary conditions.
- Comparing with the free boundary condition in the natural spline,

$$S''(a) = 0, \quad S''(b) = 0,$$

the clamped spline specifies the slope at the endpoints, i.e.,

$$S'(a) = f'(a), \quad S'(b) = f'(b).$$

- So, the clamped spline requires additional information:

x_0	x_1	x_2	$\cdots$	x_n
$f(x_0)$	$f(x_1)$	$f(x_2)$	$\cdots$	$f(x_n)$
$f'(x_0)$				$f'(x_n)$

- The rest of conditions are exactly the same as the natural splines.

We revisit Example 14, and this time we construct a clamped spline that passes through the points $(1, 2)$, $(2, 3)$, and $(3, 5)$ that has $S'(1) = 2$ and $S'(3) = 1$.

Solution (1/2)

- There are two pieces in the spline $S(x)$:

$$S_0(x) = a_0 + b_0(x-1) + c_0(x-1)^2 + d_0(x-1)^3, \quad \text{on } [1,2]$$
$$S_1(x) = a_1 + b_1(x-2) + c_1(x-2)^2 + d_1(x-2)^3, \quad \text{on } [2,3].$$

- Most conditions (6 out of 8) are the same as the natural spline,

$$f(1) = 2 \implies a_0 = 2, \quad S_0(2) = 3 \implies b_0 + c_0 + d_0 = 1.$$
$$f(2) = 3 \implies a_1 = 3, \quad S_1(3) = 5 \implies b_1 + c_1 + d_1 = 2.$$
$$s_0'(2) = s_1'(2) \implies b_0 + 2c_0 + 3d_0 = b_1.$$
$$s_0''(2) = s_1''(2) \implies 2c_0 + 6d_0 = 2c_1.$$

- The clamped boundary conditions yield

$$s_0'(1) = 2 \implies b_0 = 2,$$

$$s_1'(3) = 1 \implies b_1 + 2c_1 + 3d_1 = 1.$$

- Solve the system for eight unknowns, we have

$$S(x) = \begin{cases} 2 + 2(x-1) - \dfrac{5}{2}(x-1)^2 + \dfrac{3}{2}(x-1)^3, & \text{on } [1,2], \\ 3 + \dfrac{3}{2}(x-1) + 2(x-2)^2 - \dfrac{3}{2}(x-2)^3, & \text{on } [2,3]. \end{cases}$$

Construction of a clamped cubic spline

- Define the cubic polynomial on each interval to be

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad j = 0, 1, \cdots, n-1.$$

  The coefficients $\{a_j\}$, $\{b_j\}$, and $\{d_j\}$ are define as (3.1) and (3.2).

- For $j = 1, 2, \cdots, n-1$ we have the following equations for c_j

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}).$$

- In addition, we have the clamped boundary conditions:

$$2h_0c_0 + h_0c_1 = \frac{3}{h_0}(a_1 - a_0) - 3f'(a)$$

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1})$$

- The clamped spline defined on $a = x_0 < x_1 < \cdots < x_n = b$ is unique. $\{c_n\}$ satisfies the linear system $A\mathbf{x} = \mathbf{b}$ where

$$A = \begin{bmatrix} 2h_0 & h_0 & 0 & \cdots & \cdots & 0 \\ h_0 & 2(h_0+h_1) & h_1 & & & \\ 0 & h_1 & 2(h_1+h_2) & h_2 & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ & & & h_{n-2} & 2(h_{n-2}+h_{n-1}) & h_{n-1} \\ 0 & \cdots & \cdots & 0 & h_{n-1} & 2h_{n-1} \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} \frac{3}{h_0}(a_1 - a_0) - 3f'(a) \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \end{bmatrix}, \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}$$

Matlab File for Clamped Spline Coefficient

```
function [a,b,c,d] = clamped_spline_coef(dataX,dataY,dFa,dFb)
%% Cubic Spline (Clamped Boundary) Interpolation
%  On each sub-interval, generate four coefficients aj, bj, cj, and dj
%       Sj(x) = aj + bj(x-xj) + cj(x-xj)^2 + dj(x-xj)^3.
%
%  Inputs: dataX dataY are row vectors of same dimension.
%  Output: a is the collum vector that stores {aj} for j=1,2,...,n+1
%          b is the collum vector that stores {bj} for j=1,2,...,n+1
%          c is the collum vector that stores {cj} for j=1,2,...,n+1
%          d is the collum vector that stores {dj} for j=1,2,...,n+1

%% Initialize h
n = length(dataX) - 1;
h = zeros(1,n);
for j = 1:n
    h(j) = dataX(j+1) - dataX(j);
end

%% Prepare for Matrix A
A = zeros(n+1,n+1);
A(1,1) = 2*h(1); A(1,2) = h(1); % Clamped Boundary Condition
A(n+1,n) = h(n); A(n+1,n+1) = 2*h(n); % Clamped Boundary Condition
for j = 2:n
    A(j,j-1) = h(j-1);
    A(j,j+1) = h(j);
    A(j,j) = 2*(h(j-1)+h(j));
end
%% Prepare for vector bb
bb = zeros(n+1,1);
bb(1) = 3/h(j)*(dataY(2)-dataY(1)) - 3*dFa; % Clamped Boundary Condition
bb(n+1) = 3*dFb - 3/h(n)*(dataY(n+1)-dataY(n)); % Clamped Boundary Condition
for j = 2:n
    bb(j) = 3/h(j)*(dataY(j+1)-dataY(j)) - 3/h(j-1)*(dataY(j)-dataY(j-1));
end
%% Solve A*c = bb
c = A\bb;
%% Find a,b,d.
a = reshape(dataY(1:n+1),n+1,1);
b = zeros(n,1);
d = zeros(n,1);
for j = 1:n
    b(j) = 1/h(j)*(a(j+1)-a(j)) - h(j)/3*(2*c(j)+c(j+1));
    d(j) = (c(j+1) - c(j))/(3*h(j));
end
a(n+1) = [];
c(n+1) = [];
```

Matlab File for Clamped Spline Interpolation

```
function y = clamped_spline(dataX,dataY,dFa,dFb,x)
%% Cubic Spline (Clamped Boundary) Interpolation
%  On each sub-interval, generate four coefficients aj, bj, cj, and dj
%       Sj(x) = aj + bj(x-xj) + cj(x-xj)^2 + dj(x-xj)^3.
%
%  Inputs: dataX dataY are row vectors of same dimension.
%          x is the query point(s).
%  Output: y is the value of spline interpolation y=S(x) at query points x

%% Generate Spline Coefficient [ai,bi,ci,di] on each interval.
[a,b,c,d] = clamped_spline_coef(dataX,dataY,dFa,dFb);

%% Evaluate the query points
y = zeros(size(x));
for n = 1:length(x)
    for j = 1:length(dataX)-1
        if dataX(j) <= x(n) && dataX(j+1) >= x(n)
            k = j;
            break
        end
    end
    xk = dataX(k);
    y(n) = a(k) + b(k)*(x(n)-xk) + c(k)*(x(n)-xk)^2 + d(k)*(x(n)-xk)^3;
end
```

We revisit the spline interpolation of $f(x) = e^x$ at the following points

x	0	1	2	3
$f(x)$	1	e	e^2	e^3.

This time we use the clamped spline with the additional information $f'(0) = 1$, and $f'(3) = e^3$. Then, compare the accuracy with the natura spline interpolation.

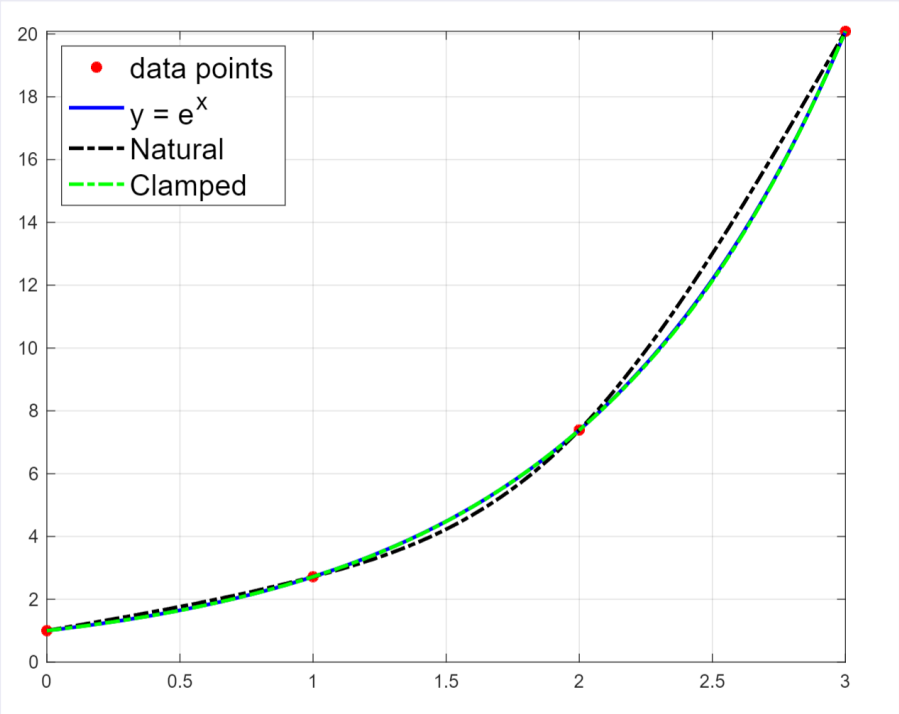
```
% ex 3.4.2
clc
dataX = [0,1,2,3];
dataY = [exp(0),exp(1),exp(2),exp(3)];

%% generate coefficient of natural spline
[a,b,c,d] = natural_spline_coef(dataX,dataY);
[a2,b2,c2,d2] = clamped_spline_coef(dataX,dataY,exp(0),exp(3));
disp('Natural Spline')
disp('    a         b         c         d')
disp('-----------------------------------------')
disp([a,b,c,d])
disp('Clamped Spline')
disp('    a         b         c         d')
disp('-----------------------------------------')
disp([a2,b2,c2,d2])

%% query point x
x = 0:0.01:3;
y = natural_spline(dataX,dataY,x);
y2 = clamped_spline(dataX,dataY,exp(0),exp(3),x);

%% plot
figure(1)
plot(dataX,dataY,'r*','linewidth',2)
hold on
plot(x,exp(x),'b-','linewidth',2)
plot(x,y,'k-.','linewidth',2)
plot(x,y2,'g-.','linewidth',2)
lgd = legend('data points','y = e^x','Natural','Clamped');
hold off
grid on
lgd.FontSize = 16;
lgd.Location = 'NorthWest';
```

- The natural spline is

$$S(x) = \begin{cases} 1.0000 + 1.4660x + 0.2523x^3, & \text{on } [0,1], \\ 2.7183 + 2.2229(x-1) + 0.7569(x-1)^2 + 1.6911(x-1)^3, & \text{on } [1,2], \\ 7.3891 + 8.8098(x-2) + 5.8301(x-2)^2 - 1.9434(x-2)^3, & \text{on } [2,3]. \end{cases}$$

- The clamped spline is

$$S(x) = \begin{cases} 1.0000 + 1.0000x + 0.4447x^2 + 0.2736x^3, & \text{on } [0,1], \\ 2.7183 + 2.7102(x-1) + 1.2655(x-1)^2 + 0.6951(x-1)^3, & \text{on } [1,2], \\ 7.3891 + 7.3265(x-2) + 3.3509(x-2)^2 + 2.0191(x-2)^3, & \text{on } [2,3]. \end{cases}$$



- From the plot, we can see that the clamped spline is more accurate than the natural spline.
- This is not surprise since the boundary conditions for the clamped spline are exact.

Theorem 18 (Error bound for Clamped Cubic Splines).

Let $f \in C^4[a,b]$ *with*

$$\max_{a \le x \le b} |f^{(4)}(x)| = M.$$

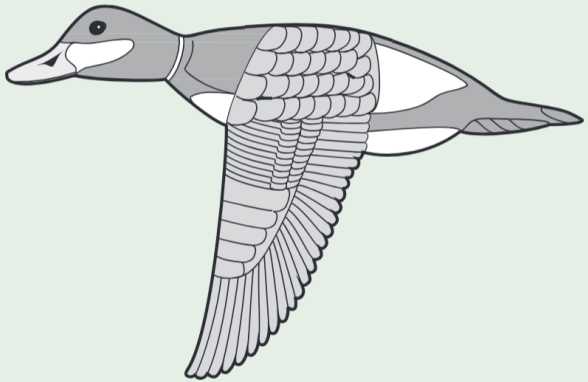
If $S(x)$ *is the unique clamped cubic spline interpolation to* f *with respect to the nodes* $a = x_0 < x_1 < \cdots < x_n = b$*, then for all* $x \in [a,b]$*,*

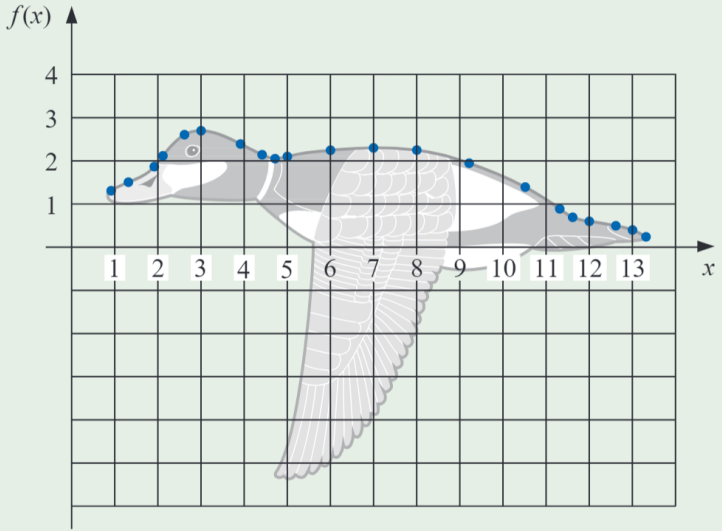
$$|f(x) - S(x)| \le \frac{5M}{384} \max_{0 \le j \le n-1} (x_{j+1} - x_j)^4.$$

Remark

- A fourth-order error bound also holds in the case of natural spline interpolation, but it is more difficult to express.
- There are other cubic spline interpolations that do not require the derivative of f. For example, the popular "not-a-knot spline" requires that the third-order derivative $S'''(x)$ is continuous at x_1 and x_{n-1}.

In this example, we approximate the top profile of the duck using cubic spline interpolation.





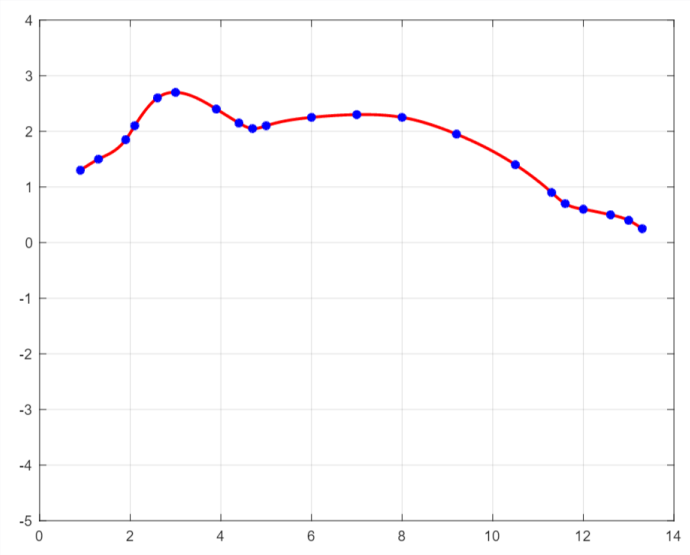
In general, the more points we use, the better approximation we can expect. We choose 21 data points as depicted above and shown below in the table. Note that more points are placed where the curve is changing more rapidly.

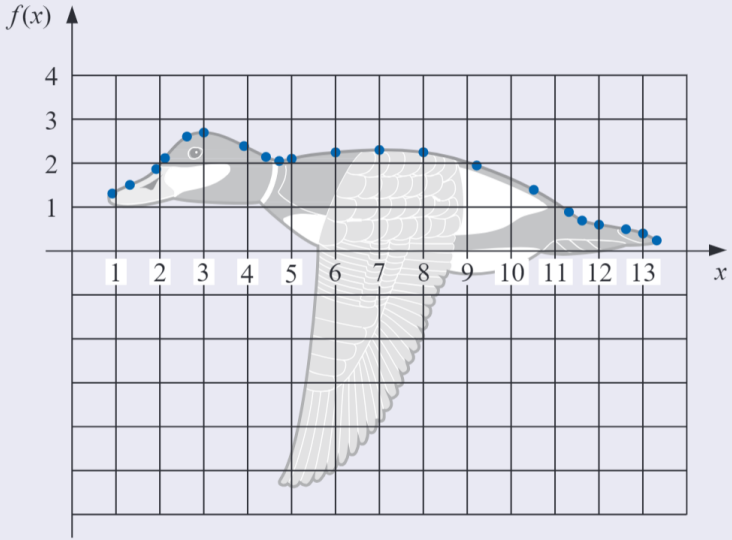
x	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0	12.6	13.0	13.3
$f(x)$	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6	0.5	0.4	0.25

- Since we don’t have derivative information, we use the natural spline interpolation.
- We write a Matlab driver file for this example.

```
% ex 3.5.illustration
clear; clc;

%% Input
dataX = [0.9 1.3,1.9 2.1 2.6 3.0 3.9 4.4 4.7 5.0 6.0 7.0 8.0 ...
    9.2 10.5 11.3 11.6 12.0 12.6 13.0 13.3];
dataY = [1.3 1.5 1.85 2.1 2.6 2.7 2.4 2.15 2.05 2.1 2.25 2.3 2.25 ...
    1.95 1.4 0.9 0.7 0.6 0.5 0.4 0.25];
[a,b,c,d] = natural_spline_coef(dataX,dataY); % coefficent of cubic spline

%% Display Outputs
disp('-----------------------------------------------------')
disp(' j      a           b           c           d')
disp('-----------------------------------------------------')
formatSpec = '%2i    % .5f    % .5f    % .5f    % .5f   \n';
fprintf(formatSpec,[(0:length(dataX)-2)',a,b,c,d]')

%% Plot Spline
x = 0.9:0.01:13.3; % query points
y = natural_spline(dataX,dataY,x);
figure(1)
clf;
plot(x,y,'r-','linewidth',2)
hold on
plot(dataX,dataY,'b*','linewidth',2)
axis([0,14,-6,4])
grid on
hold off
```

- Plotting the natural spline interpolation, we observe that the spline curve is accurately recovers the top profile of the duck.



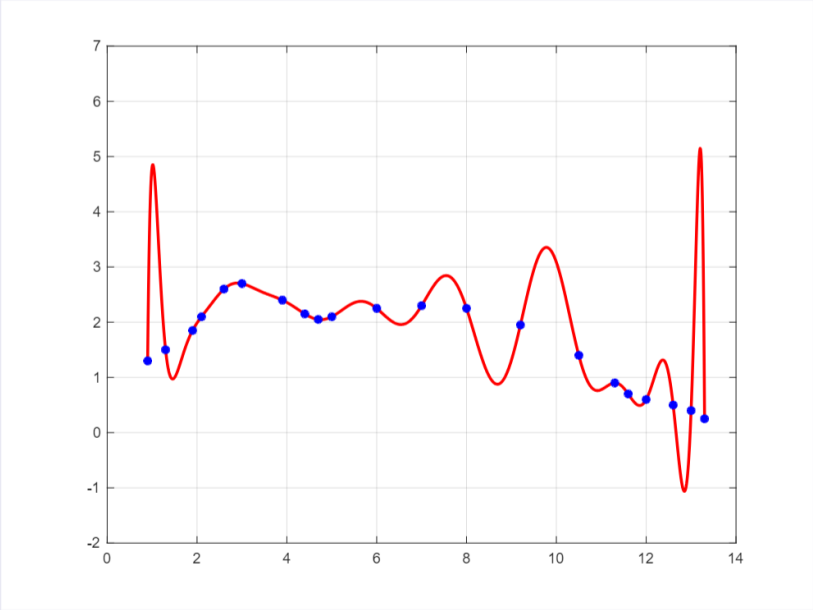


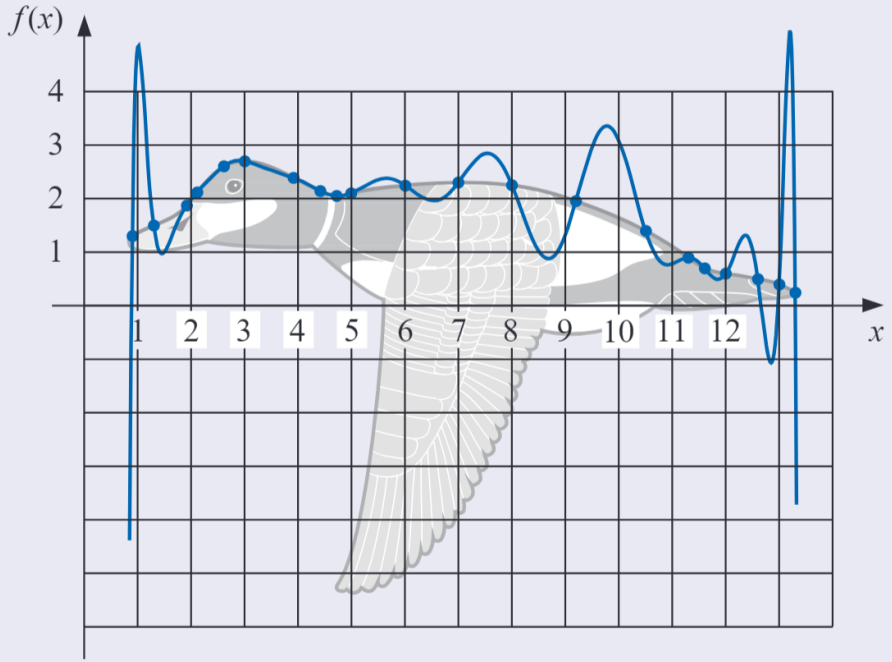
- To use a clamped spline we would need derivative approximations for the endpoints. Even if these approximations were available, we could expect little improvement because of the close agreement of the natural cubic spline to the curve of the top profile.

For comparison, we also use Lagrange Interpolation.

```
% Duck Interpolation Comparison
clear; clc;

%% Input
dataX = [0.9 1.3,1.9 2.1 2.6 3.0 3.9 4.4 4.7 5.0 6.0 7.0 8.0 ...
    9.2 10.5 11.3 11.6 12.0 12.6 13.0 13.3];
dataY = [1.3 1.5 1.85 2.1 2.6 2.7 2.4 2.15 2.05 2.1 2.25 2.3 2.25 ...
    1.95 1.4 0.9 0.7 0.6 0.5 0.4 0.25];

%% Find Spline and Lagrange Interpolation
x = 0.9:0.01:13.3; % query points
y = natural_spline(dataX,dataY,x);
yy = LagrangeInterpolation(dataX,dataY,x);

%% Plot Interpolations
figure(1)
clf;
plot(dataX,dataY,'b*','linewidth',2)
hold on
plot(x,y,'r-','linewidth',2)
plot(x,yy,'k-','linewidth',2)
axis([0,14,-2,7])
hold off
grid on
lgd = legend('Data Point','Natural Spline','Lagrange');
lgd.FontSize = 16;
lgd.Location = 'NorthWest';
```

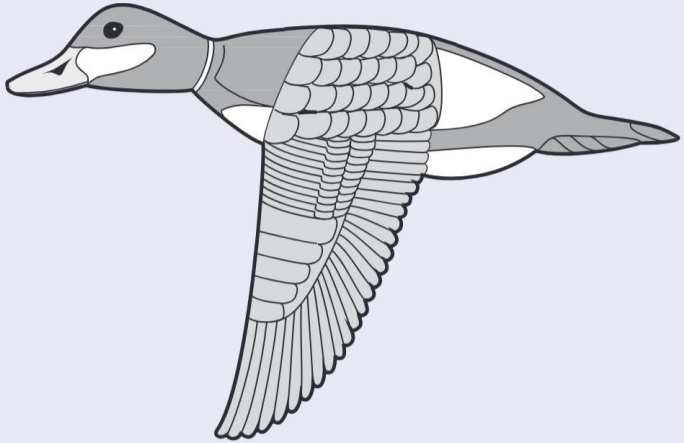
- Plotting the Lagrange interpolation, we observe that the 20th-degree polynomial oscillates wildly. It produces a very strange illustration of the back of a duck.





- This example shows the superiority of the cubic spline interpolation.

Remark



- Constructing a cubic spline to approximate the lower profile of the duck would be more difficult since the curve for this portion cannot be expressed as a function of x, and at certain points the curve does not appear to be smooth.

- Parametric curves can be used

Figure 3.11 shows a ruddy duck in flight. To approximate the top profile of the duck, we have chosen points along the curve through which we want the approximating curve to pass. Table 3.18 lists the coordinates of 21 data points relative to the superimposed coordinate system shown in Figure 3.12. Notice that more points are used when the curve is changing rapidly than when it is changing more slowly.

Table 3.18

x	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0	12.6	13.0	13.3
$f(x)$	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6	0.5	0.4	0.25

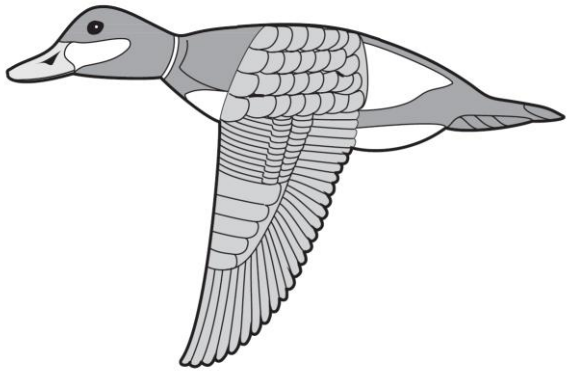
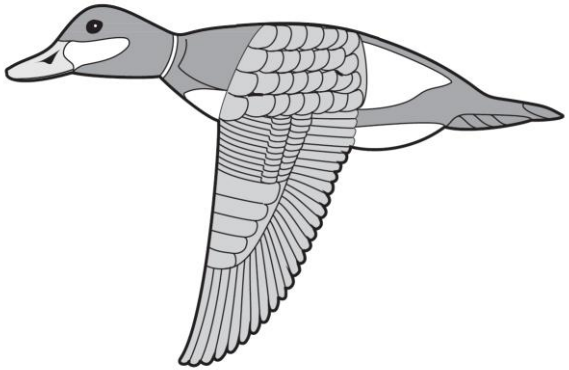


Figure 3.11 shows a ruddy duck in flight. To approximate the top profile of the duck, we have chosen points along the curve through which we want the approximating curve to pass. Table 3.18 lists the coordinates of 21 data points relative to the superimposed coordinate system shown in Figure 3.12. Notice that more points are used when the curve is changing rapidly than when it is changing more slowly.

Table 3.18

x	0.9	1.3	1.9	2.1	2.6	3.0	3.9	4.4	4.7	5.0	6.0	7.0	8.0	9.2	10.5	11.3	11.6	12.0	12.6	13.0	13.3
$f(x)$	1.3	1.5	1.85	2.1	2.6	2.7	2.4	2.15	2.05	2.1	2.25	2.3	2.25	1.95	1.4	0.9	0.7	0.6	0.5	0.4	0.25



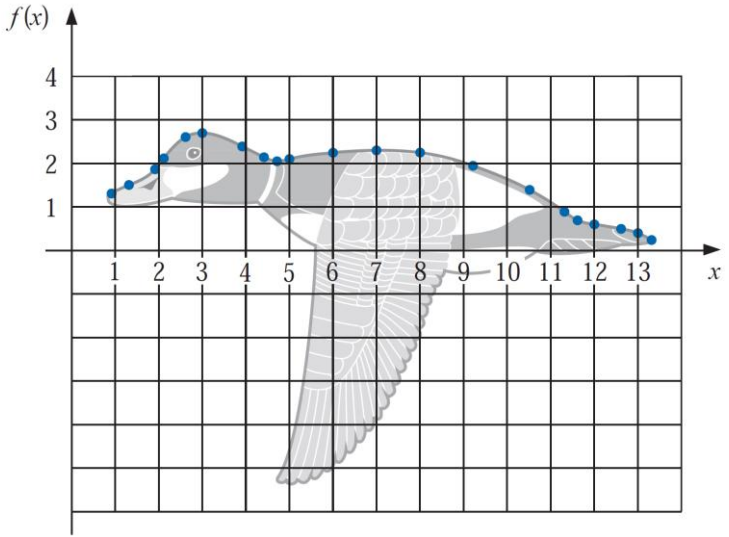
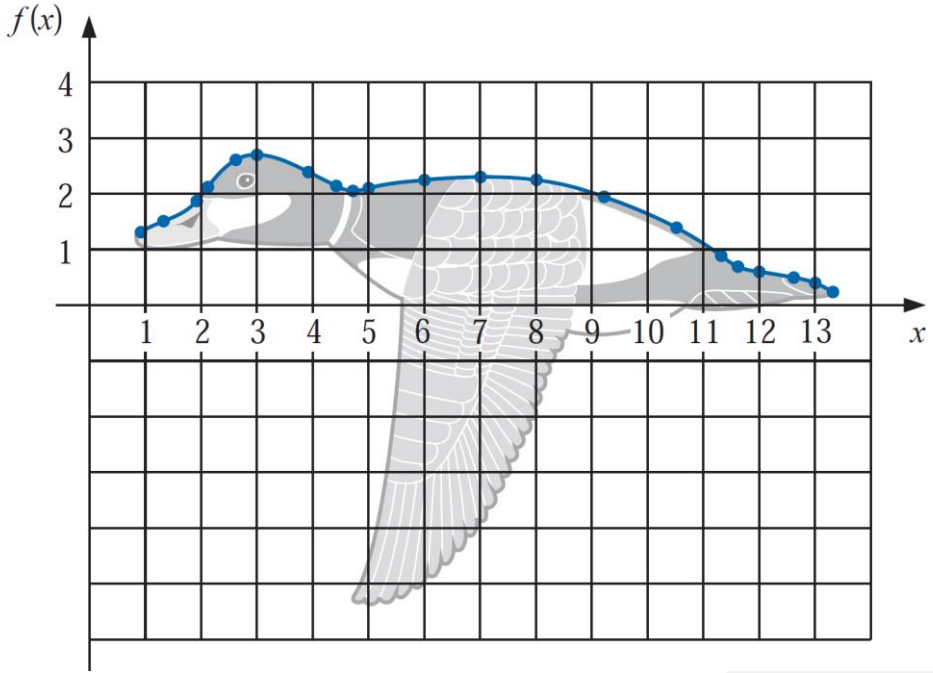
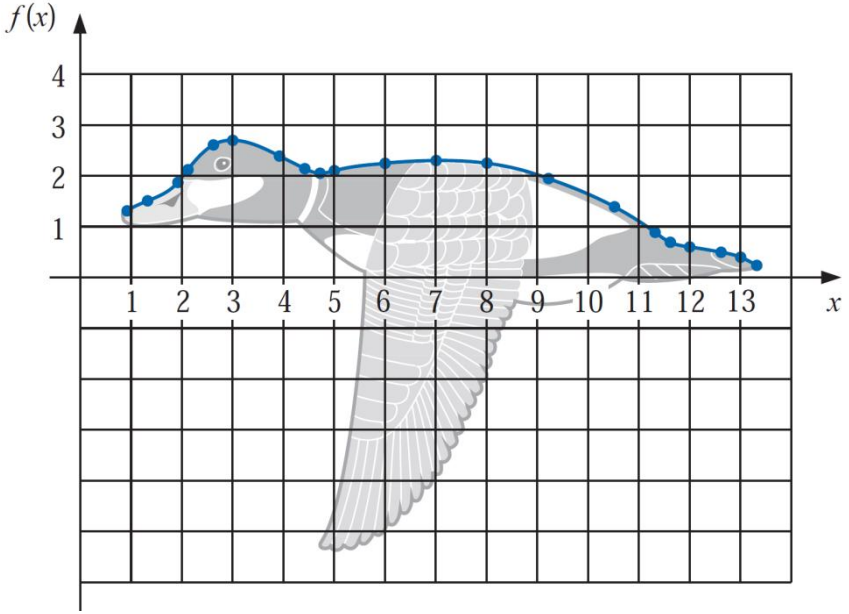


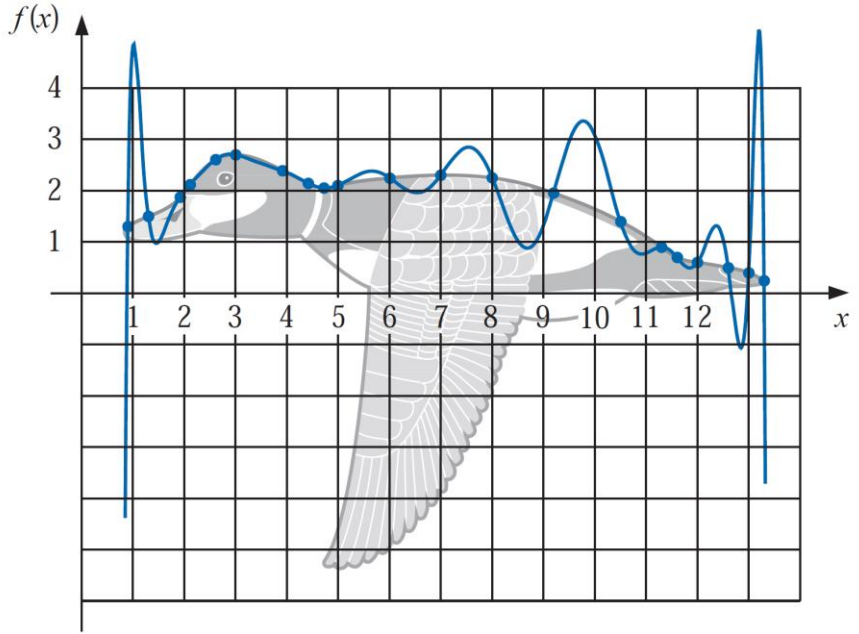
Table 3.19

j	x_j	a_j	b_j	c_j	d_j
0	0.9	1.3	5.40	0.00	−0.25
1	1.3	1.5	0.42	−0.30	0.95
2	1.9	1.85	1.09	1.41	−2.96
3	2.1	2.1	1.29	−0.37	−0.45
4	2.6	2.6	0.59	−1.04	0.45
5	3.0	2.7	−0.02	−0.50	0.17
6	3.9	2.4	−0.50	−0.03	0.08
7	4.4	2.15	−0.48	0.08	1.31
8	4.7	2.05	−0.07	1.27	−1.58
9	5.0	2.1	0.26	−0.16	0.04
10	6.0	2.25	0.08	−0.03	0.00
11	7.0	2.3	0.01	−0.04	−0.02
12	8.0	2.25	−0.14	−0.11	0.02
13	9.2	1.95	−0.34	−0.05	−0.01
14	10.5	1.4	−0.53	−0.10	−0.02
15	11.3	0.9	−0.73	−0.15	1.21
16	11.6	0.7	−0.49	0.94	−0.84
17	12.0	0.6	−0.14	−0.06	0.04
18	12.6	0.5	−0.18	0.00	−0.45
19	13.0	0.4	−0.39	−0.54	0.60
20	13.3	0.25			





Natural Cubic Spline interpolation



Lagrange interpolation

NAZARBAYEV
UNIVERSITY